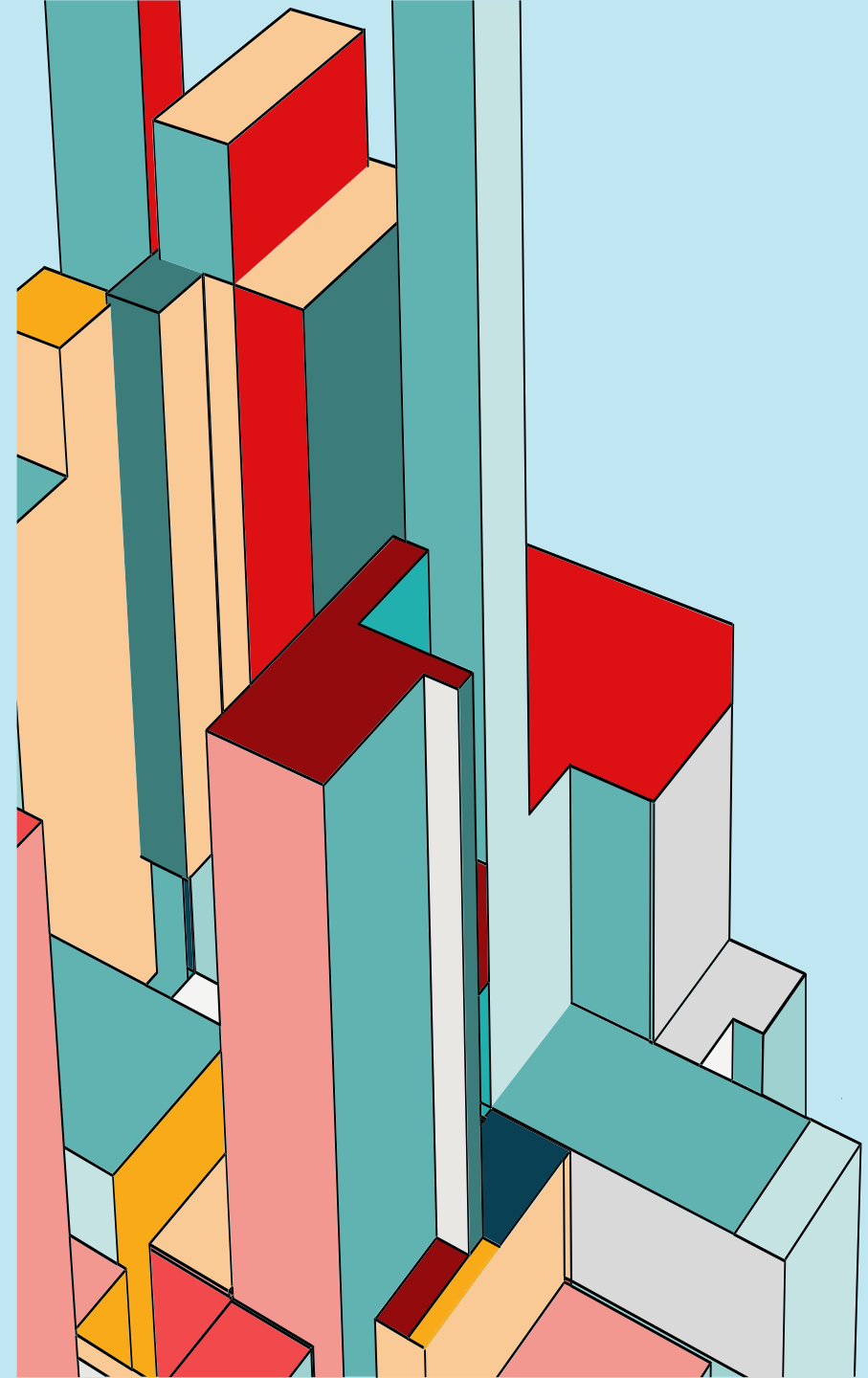


SOFTWARE ARCHITECTURE • ADAS / HPC ECUS

AUTOSAR Adaptive Platform

An architecture walkthrough – functional clusters, service-oriented ara::com, methodology & manifests, and the C++ design rules behind it.



What this deck covers

Built from the AUTOSAR R22-11 Adaptive Platform explanatory & methodology specifications

01 Why an Adaptive Platform

Technology drivers, AP characteristics, CP vs AP

02 Architecture & Functional Clusters

ARA stack, logical/physical views, the FC landscape

03 Runtime & Safety

Execution Mgmt, State Mgmt, Platform Health Mgmt

04 Communication (SOA / ara::com)

Proxy/Skeleton, methods-events-fields, discovery, bindings

05 Storage · Security · OTA · Diagnostics

Persistency, Crypto/IAM/IdsM/Firewall, UCM, UDS/DoIP

06 Methodology & Manifests

Design vs deployment, the three manifests, SW clusters

07 Design Principles

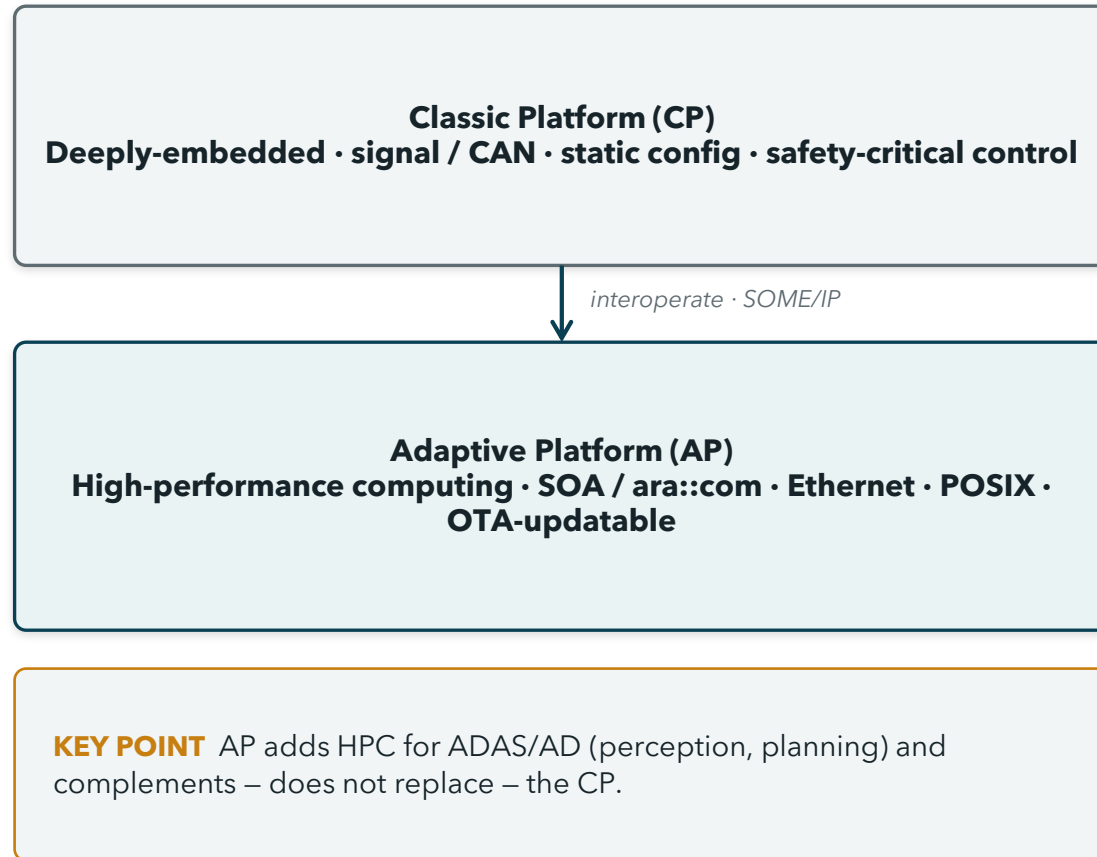
C++14 guidelines, ara::core error handling, SOLID + ADP

1

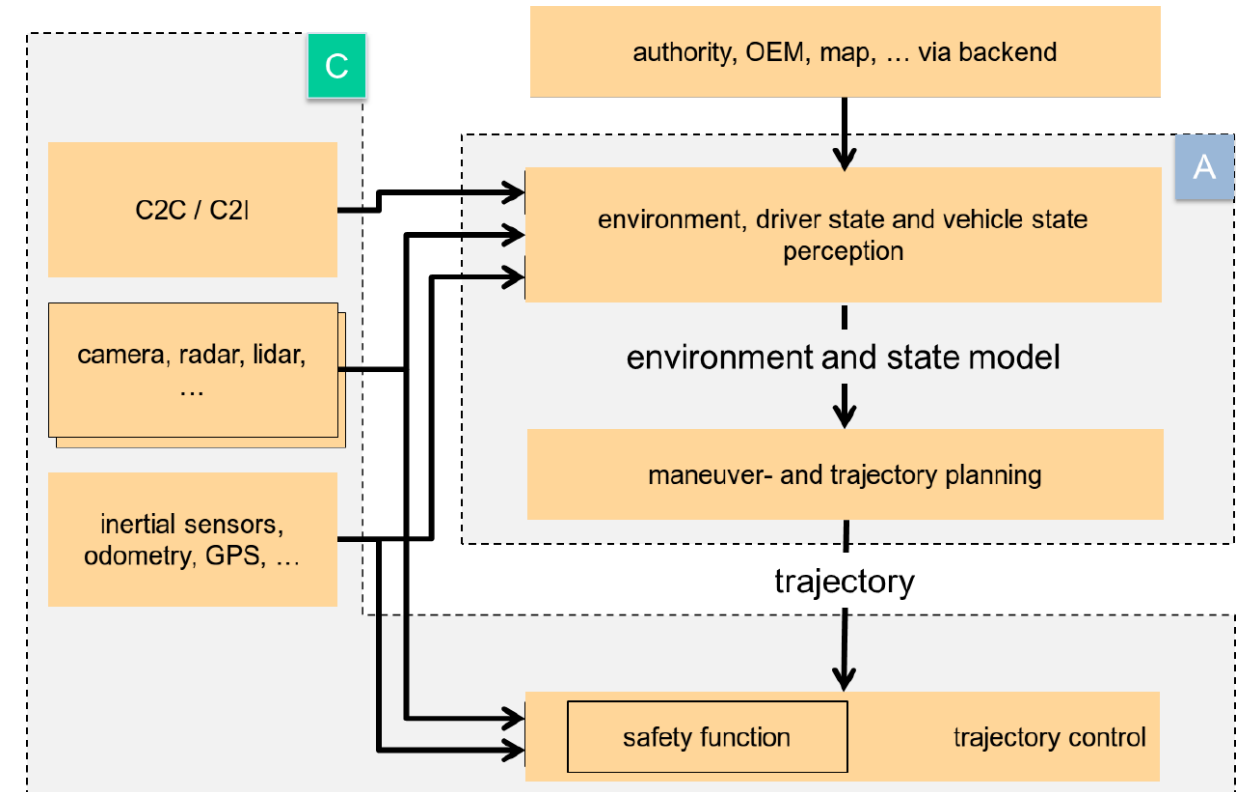
Why a second platform?

TECHNICAL SCOPE

CP vs AP – complementary platforms



Exemplary interactions of AP and CP



A = Adaptive Platform partition · C = Classic Platform partition

2

Two technology drivers

TECHNICAL SCOPE

Ethernet

Ever-increasing on-vehicle bandwidth led to switched Ethernet:

- Higher bandwidth than CAN; efficient long messages
- Point-to-point & switched topologies
- Enables service-oriented comms (SOME/IP)

CP, though it can run Ethernet, was optimized for legacy buses and cannot fully exploit it.

Processors

Intelligence demands more than multicore:

- Manycore (tens-hundreds of cores), GPGPU, FPGA, accelerators
- Heterogeneous computing → best performance-per-watt
- Network-on-Chip (NoC) inter-core links

Pollack's Rule: frequency cannot scale endlessly – the only path is parallel execution across many cores.

3

Adaptive Platform – seven characteristics

CHARACTERISTICS

C++

Language of choice for new, performance-critical algorithms

SOA

System of services; location-transparent communication

Parallel processing

Distributed compute maps to manycore / heterogeneous HW

Leverage standards

Reuse C++, POSIX, SOME/IP – don't reinvent

Safety & security

Architectural + functional + procedural measures

Planned dynamics

Dynamic behaviour, but bounded by manifests for qualification

Agile

Incrementally scalable; updatable after deployment



PART

Architecture & Functional Clusters

Logical & physical views · the ARA · the functional cluster landscape

Logical view – AUTOSAR Runtime for Adaptive Applications (ARA)

Applications sit on ARA: the standardized interfaces of Functional Clusters + ara::com

Adaptive Applications (AA)

User SWCs · Adaptive Platform Services · non-platform services

ARA – AUTOSAR Runtime for Adaptive Applications

ara::com (the only explicit comm interface) + ara:: Functional Cluster APIs · C++ language binding*

C++ Standard Library · POSIX (PSE51) API

Standardized language binding

Operating System · Machine / Hardware

manycore · GPGPU · virtualization

▼ AA code is identical whether a service is local or on a remote AP – the middleware hides the difference

4

Physical view – OS, processes, threads, machine

ARCHITECTURE

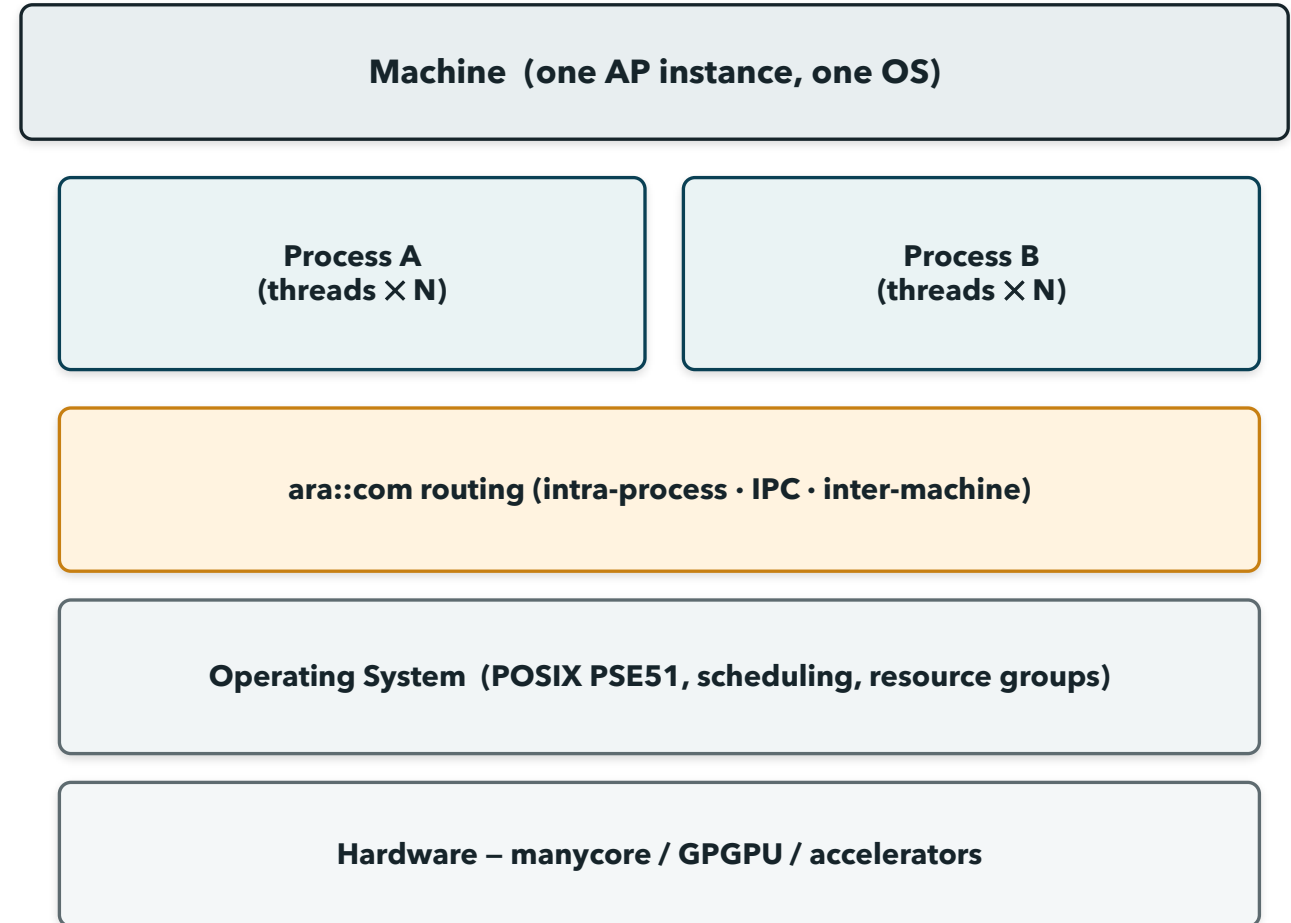
EXECUTION MODEL

An AP instance is a Machine running one OS. Applications are deployed as Processes; each may spawn multiple Threads scheduled by the OS across cores.

INTERACTIONS

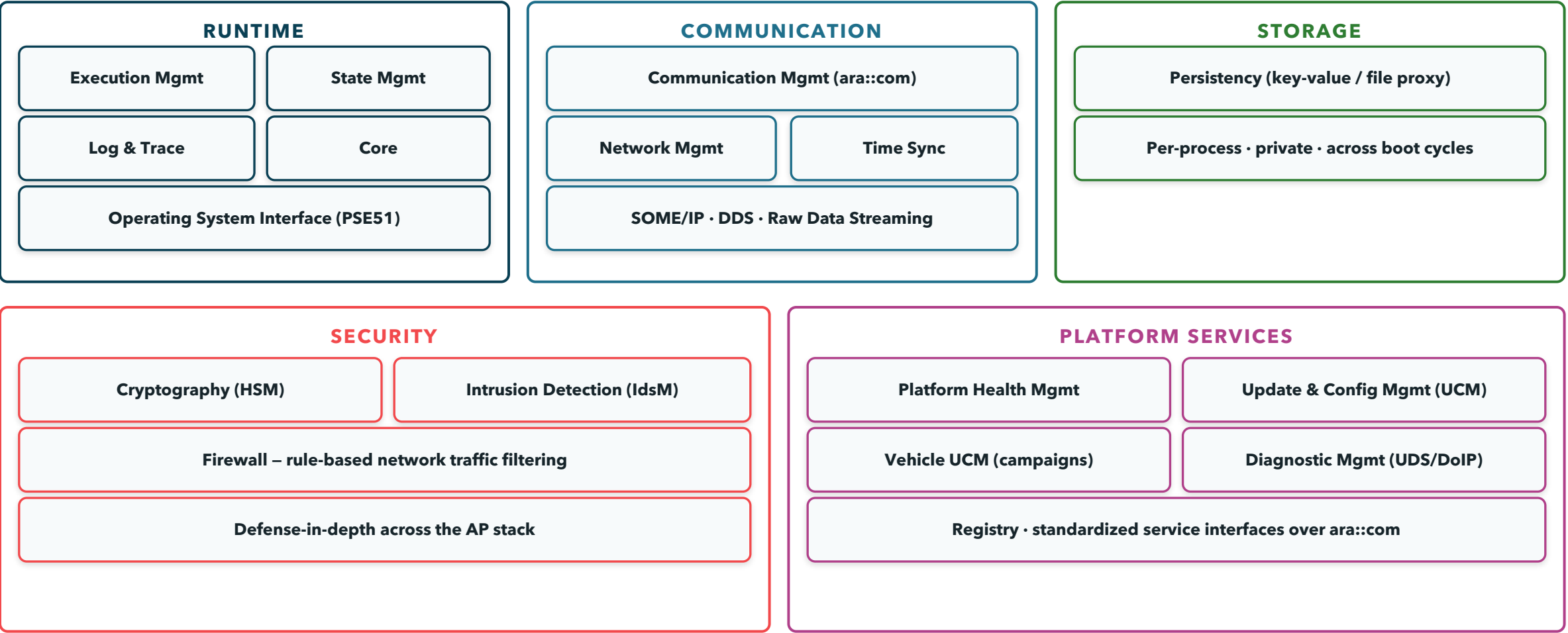
- Functional Clusters interact via defined interfaces
- ara::com handles routing regardless of deployment topology
- FCs may be co-located on one AP or distributed over many
- A Machine may be a real ECU or a virtualized partition

MANIFEST-DRIVEN Deployment of processes & services is configured in manifests, not hard-coded.



The Functional Cluster landscape

Adaptive Platform Foundation + Adaptive Platform Services, grouped by domain



Foundation FCs are typically library-based; Adaptive Platform Services are service-based (reachable over ara::com).

Source: AUTOSAR_AP_EXP_SWArchitecture, R24-11

5

Two FC implementation styles

ARCHITECTURE

Library-based

The FC functionality is linked directly into the application; the process calls the API as a local library (direct IPC where needed).

- Lower latency, tighter coupling
- Used by Adaptive Platform Foundation clusters
- e.g. Core, Log&Trace, OS interface

```
ara::core · ara::log
```

Service-based

The process uses a generated proxy library that calls Communication Management, which coordinates IPC with the server.

- Location-transparent, loosely coupled
- Recommended where flexibility matters
- Used by Adaptive Platform Services
- e.g. Diagnostics, UCM, PHM as services

```
ara::com proxy/skeleton
```



PART

Runtime & Safety

Execution Management · State Management · Platform Health Management

6

Execution Management (EM)

RUNTIME

RESPONSIBILITY

Controls Processes of the AP and applications – starts, configures and stops them per Function Group States, using OS interfaces. The OS does the actual scheduling.

ALSO HANDLES

- Entry point of the AP; started by OS at boot
- Resource limits (CPU, memory) via OS Resource Groups
- Authenticated startup – chain of trust from a Trust Anchor
- Deterministic (cyclic) execution support
- Process state reporting via ExecutionClient

NOTE EM controls startup & shutdown of the whole platform.

KEY CONCEPTS

**Function Group – a set of related Processes
(defined in the Manifest)**

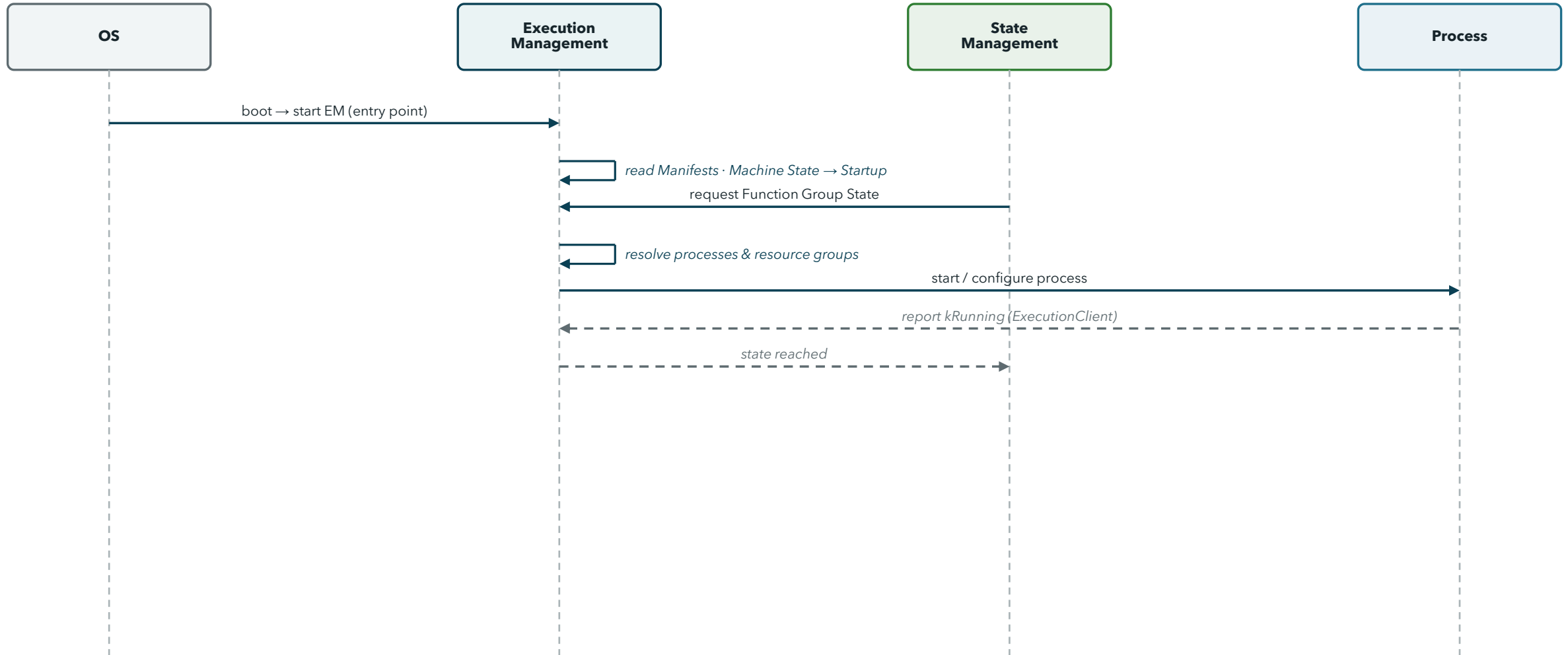
**Function Group State – the active state of a Function Group
(e.g. Off / Running / Diagnostics)**

**Machine State – the special Function Group
governing overall machine lifecycle (Startup...)**

**ExecutionClient – process reports execution state
DeterministicClient – RunWorkerPool, GetRandom
for cyclic, reproducible execution**

EM – startup & state activation

How a target state becomes a set of running processes



EM translates the desired Function Group States (from SM) into concrete processes started via the OS – bounding CPU/memory through Resource Groups.

7

State Management (SM)

RUNTIME

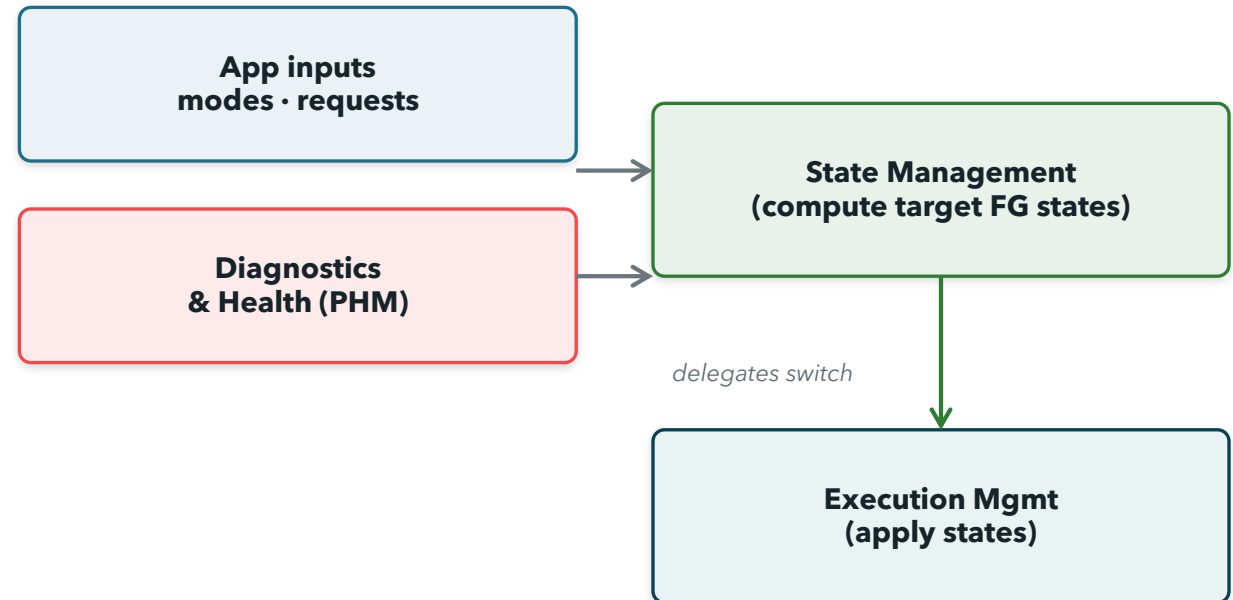
RESPONSIBILITY

Determines the desired target state of the ARA from application-specific inputs – the set of active Function Group States across all Function Groups.

HOW IT WORKS

- Collects inputs (modes, requests, diagnostics, health)
- Computes target Function Group States
- Delegates the actual switch to Execution Management
- Highly project-specific – the platform's 'brain'

DESIGN SM is unique: largely defined by the integrator/OEM, not fully standardized.



8

Platform Health Management (PHM)

SAFETY · ISO 26262

RESPONSIBILITY

Supervises Processes in safety-critical setups via reported Checkpoints, and reports failures to State Management. PHM also drives the hardware Watchdog – which in turn supervises PHM.

RECOVERY

- Failures routed to SM → recovery actions
- Watchdog triggers reset if PHM itself fails
- Supervises SM & EM – the recovery path itself

BASIS All supervision is driven by ReportCheckpoint() from supervised entities.

Alive Supervision

Entity runs neither too often nor too rarely

Deadline Supervision

Steps execute within min/max time bounds

Logical Supervision
Control flow matches the designed sequence of checkpoints

Watchdog ⇌ PHM → State Management (recovery)



PART

Communication — SOA & ara::com

Proxy/Skeleton · methods, events, fields, triggers · service discovery · bindings

9

Communication Management (CM)

COMMUNICATION

RESPONSIBILITY

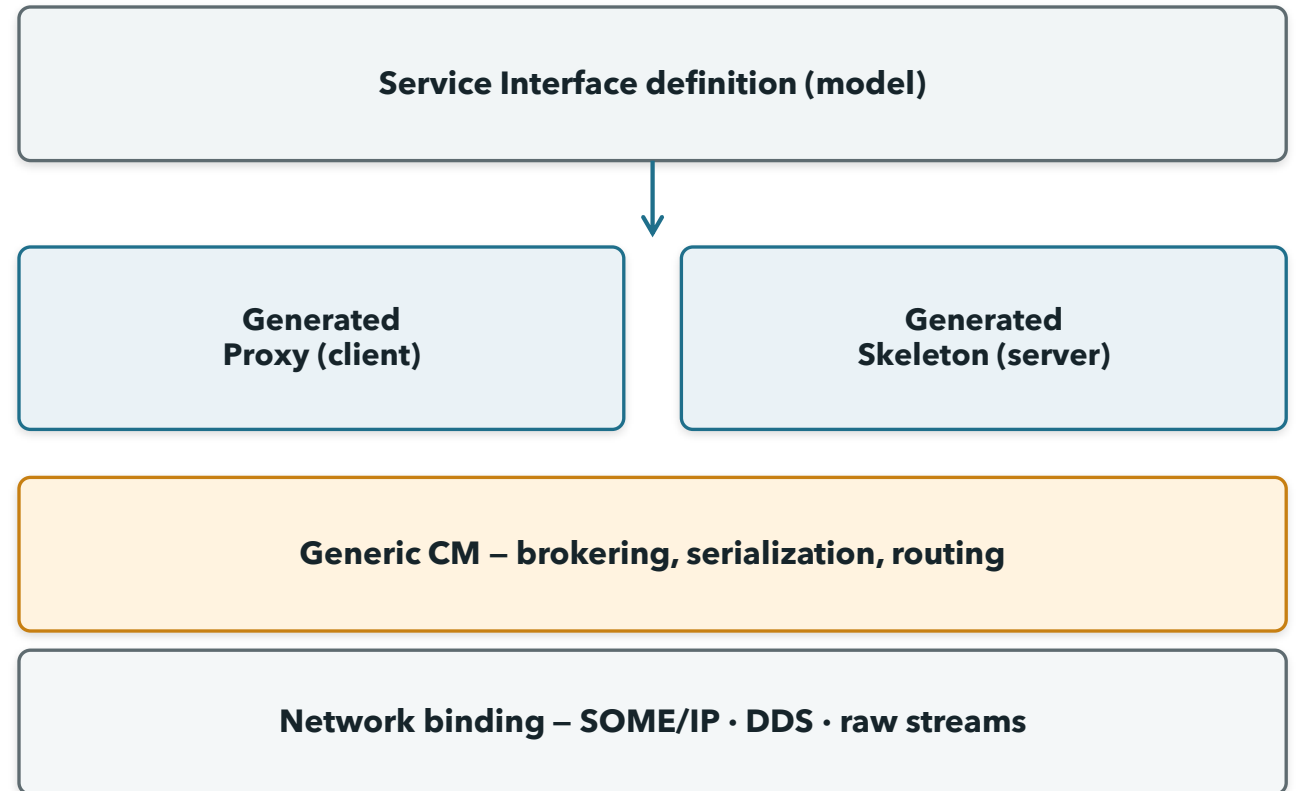
Handles all service-oriented and raw communication between applications – intra-process, inter-process and inter-machine (incl. CP and third-party platforms).

STRUCTURE

- Generic part: brokering + configuration
- Generated part: per-service Proxies & Skeletons
- Comm paths established at design / startup / run time
- ara::com is the only explicit ARA communication interface

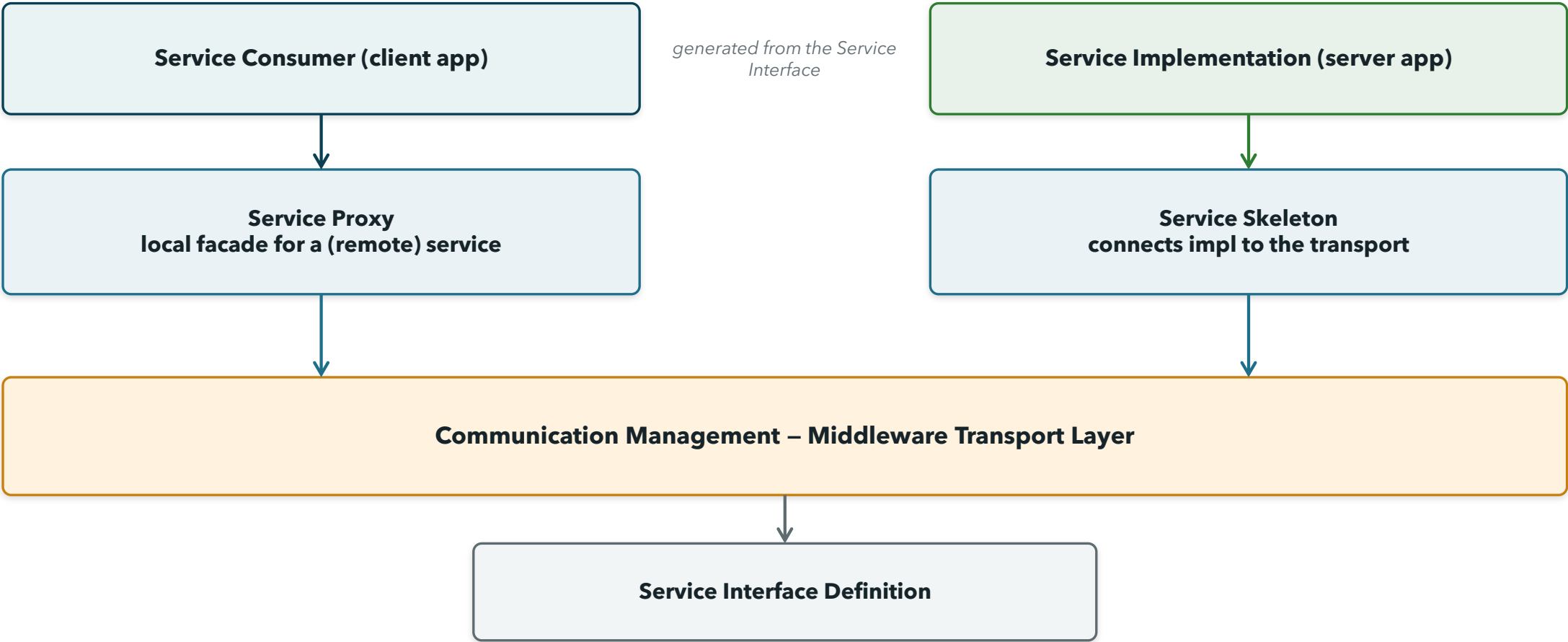
TRANSPARENCY Same application code whether the peer is local or remote.

CM = GENERIC BROKER + GENERATED ENDPOINTS



The Proxy / Skeleton pattern

From one Service Interface definition, two artifacts are generated



Consumer code talks to the local proxy; CM propagates calls to the remote skeleton and back. A class-binding subclasses the skeleton.

Source: AUTOSAR_AP_EXP_PlatformDesign, R24-11 (Communication Management)

10

ara::com — four means of communication

COMMUNICATION

Methods

Where client calls a function executed on the server

Roles may return value · `ara::core::Future` / `Promise` · `kEvent` / `kEventSingleThread` / `kPoll` · fire-and-forget

Events

Where server pushes data; clients subscribe & fetch

Roles `Subscribe()` · `GetNewSamples()` · `SamplePtr` · `SetReceiveHandler` · event-driven or polling

Fields

Where stateful value: notification + Get + Set

Roles combines event + getter/setter · always has a valid value · `GetHandler` / `SetHandler`

Triggers

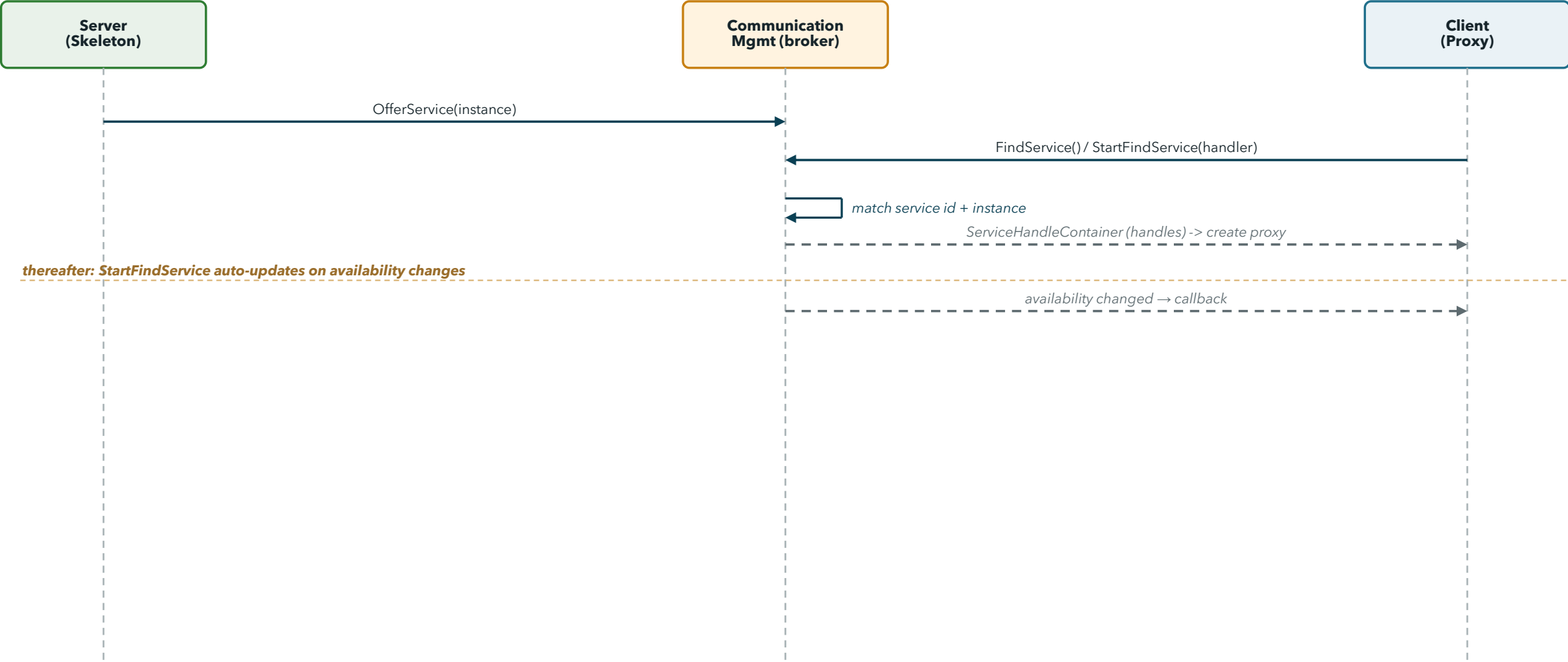
Where server signals a condition; carries no data

Roles same subscribe/notify path as events · pure notification

Precondition: server `OfferService()`; client `FindService()` / `StartFindService()` before any mechanism is used.

Service discovery – Offer / Find

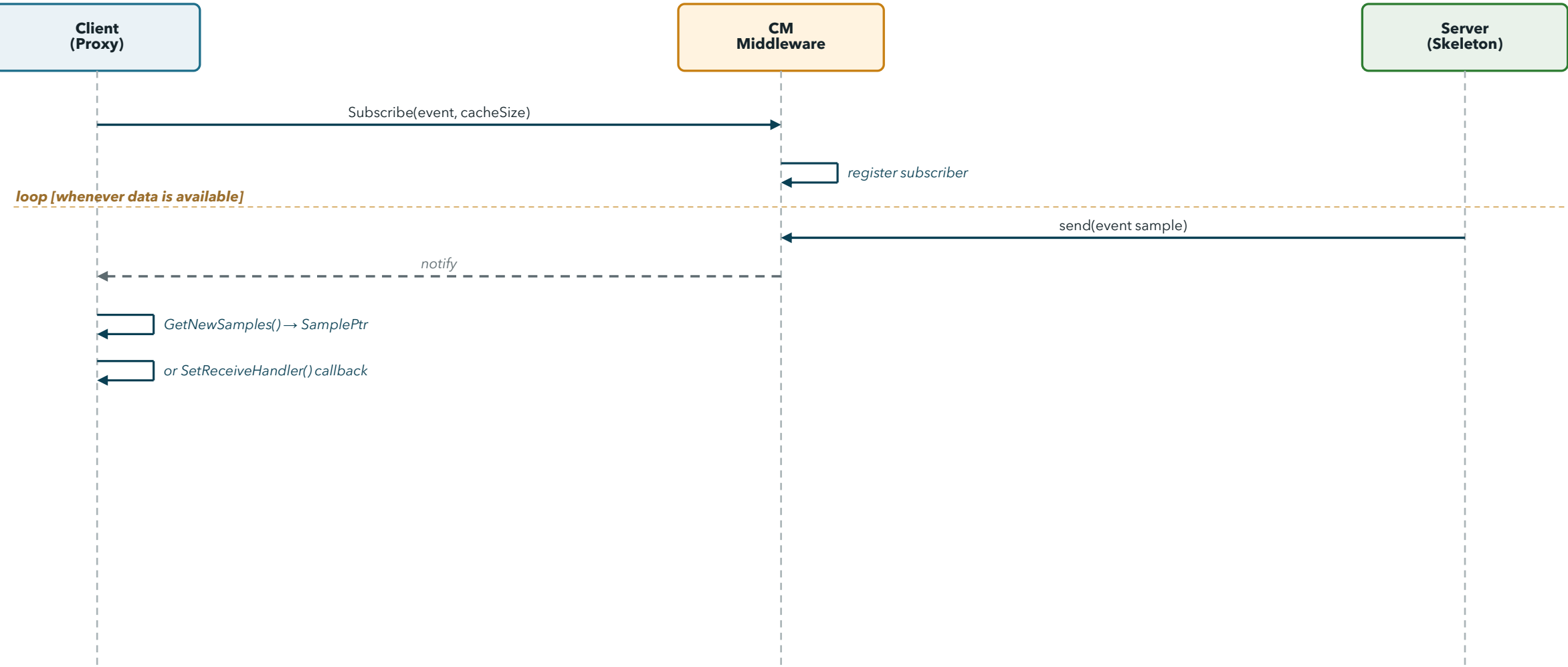
Decoupling provider and consumer at runtime



InstanceIdentifier (concrete) vs InstanceSpecifier (abstract, resolved via manifest) decouple application code from deployment.

Event & Trigger communication

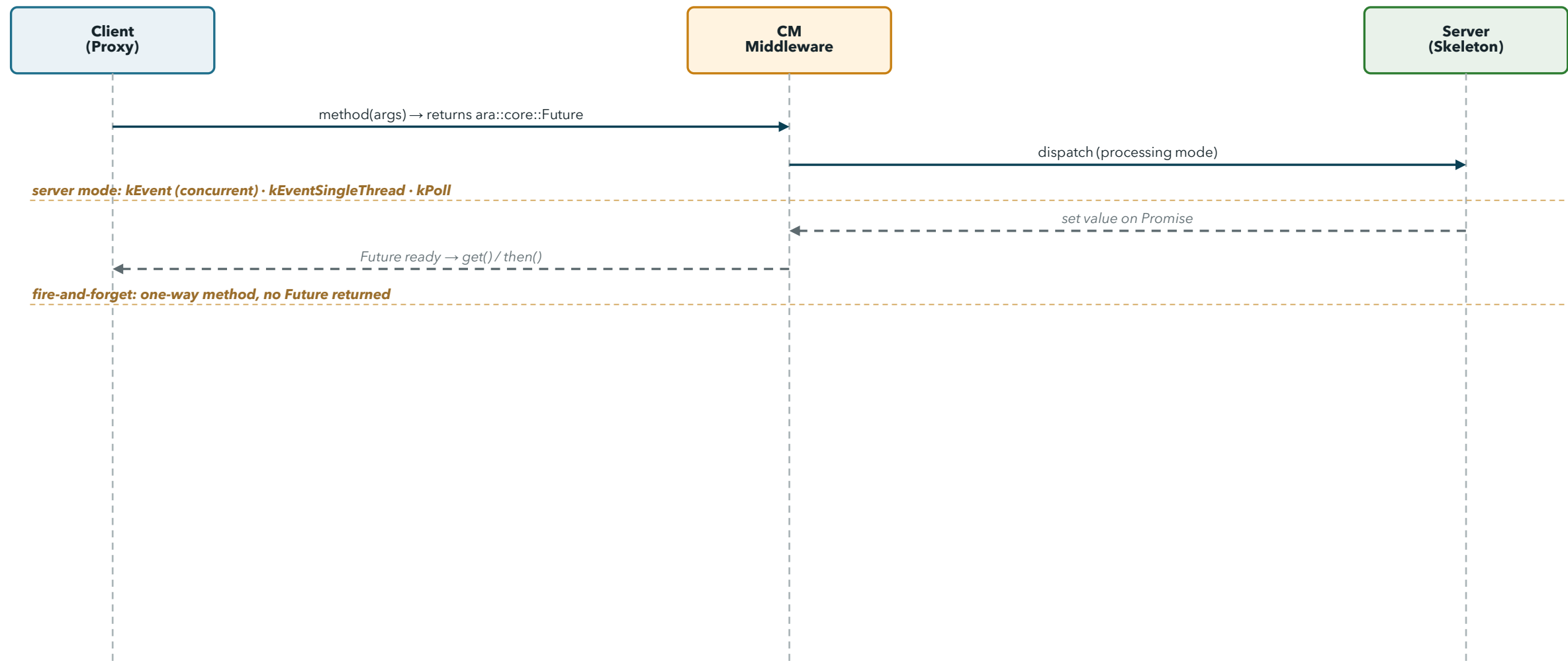
Push model with subscriber-side sample cache



Event-driven (callback on notification) vs polling (`GetNewSamples` on demand). `SamplePtr` manages sample lifetime within the configured local cache.

Method communication

Future/Promise enables non-blocking calls



Future/Promise gives non-blocking semantics; the server's processing mode controls concurrency. Results can be polled or awaited; calls can be canceled.

11

Worked example – RadarService (ADAS)

COMMUNICATION

SERVICE INTERFACE

The ara::com explanatory doc models a RadarService – the canonical ADAS example combining all four mechanisms.

TYPICAL MEMBERS

- Event: BrakeEvent / object list samples
- Method: Calibrate(...) → Future<result>
- Field: UpdateRate (Get / Set / notify)
- Trigger: condition signaling without payload

FLOW Skeleton OfferService → Proxy FindService
→ Subscribe events.

```
// Server side
RadarServiceSkeleton skel{instanceId, kEvent};
skel.OfferService();
skel.BrakeEvent.Send(sample);

// Client side
auto h = RadarServiceProxy::FindService(spec);
RadarServiceProxy proxy{h[0]};
proxy.BrakeEvent.Subscribe(10);
proxy.BrakeEvent.SetReceiveHandler([]{ /* GetNewSamples */
});
auto f = proxy.Calibrate(cfg); // Future
```

12

Language binding vs Network binding

COMMUNICATION

Language binding

Maps the service model to a programming API.

- C++ binding: generated Proxy & Skeleton classes
- Built on the C++ Standard Library
- ara::core types (Future, Result) used throughout
- Future language bindings possible

ara::com C++ API

Network binding

Maps the model to wire protocol & transport.

- SOME/IP (also used by CP) and DDS
- Service Instance Manifest holds the config
- Static or dynamic configuration
- Service contract versioning
- E2E protection; Raw Data Streaming for bulk

SOME/IP · DDS

Same application code, different binding – chosen at deployment via the Service Instance Manifest.

13

Network Management & Time Synchronization

COMMUNICATION

Network Management

Requests and queries network states for logical network handles.

- Handles map to physical or partial networks (PNC / VLAN)
- Coordinates wake-up / keep-awake / shutdown
- NetworkCurrentState field exposes status

`ara::nm`

Time Synchronization

Provides synchronized time across distributed applications & machines.

- Correlate events across a distributed system
- Trigger actions at an accurate point in time
- Essential for sensor fusion timestamps

`ara::tsync`



PART

Storage · Security · OTA · Diagnostics

Persistency · Crypto/IAM/IdsM/Firewall · Update & Config Mgmt · UDS/DoIP

14

Persistence

STORAGE

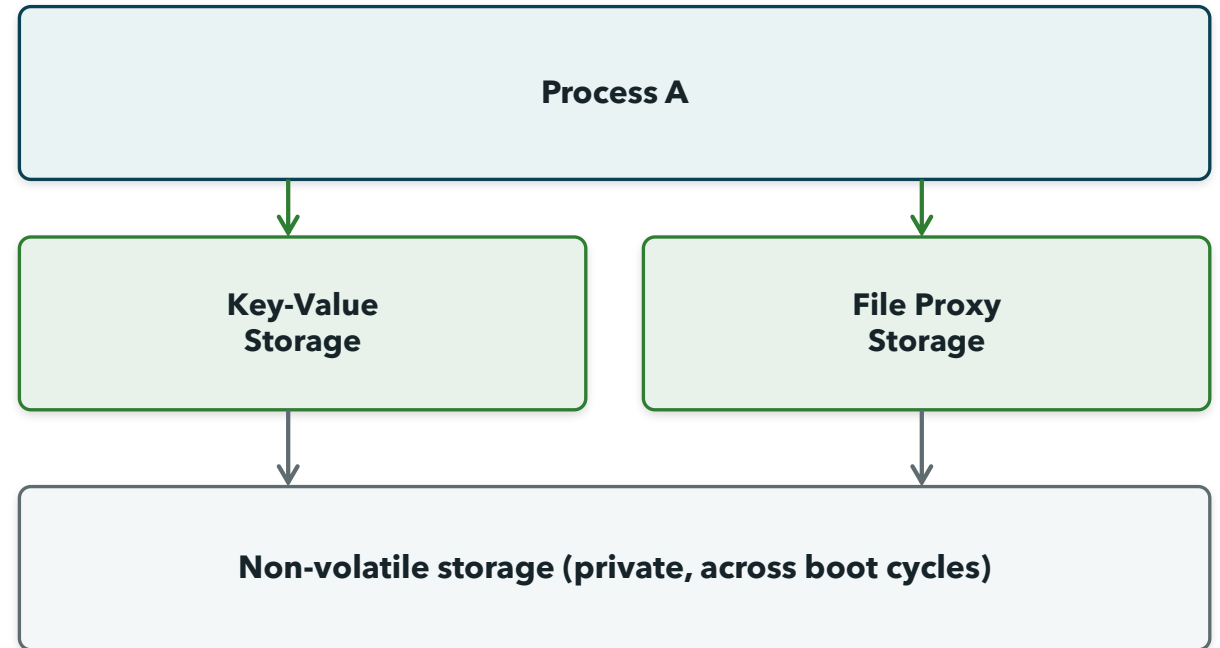
RESPONSIBILITY

Stores and retrieves information to/from non-volatile storage of a Machine, persisted across boot and ignition cycles.

RULES

- Data is always private to one Process
- No data sharing between processes (prevents a 2nd path besides CM)
- Concurrent access from threads of the same app is supported
- Key-Value storage & File-Proxy APIs

WHY PRIVATE Keeping data process-private preserves CM as the single communication channel.



15

Security – defense in depth

SECURITY

Cryptography

Where confidentiality, integrity (hashes), key mgmt, RNG

Roles encapsulates security ops in HSM · key-usage constraints (decrypt-only...)

Identity & Access Mgmt (IAM)

Where access control on Service Interfaces & FCs

Roles prevents privilege escalation · integrators verify access at deploy time

Intrusion Detection (IdsM)

Where reports security events

Roles Adaptive IDS Manager · feeds a security event pipeline

Firewall

Where filters network traffic by rules

Roles rules parsed from Manifest · engine at TCP/IP or lower · mode-dependent

16

Update & Configuration Management (UCM)

OTA

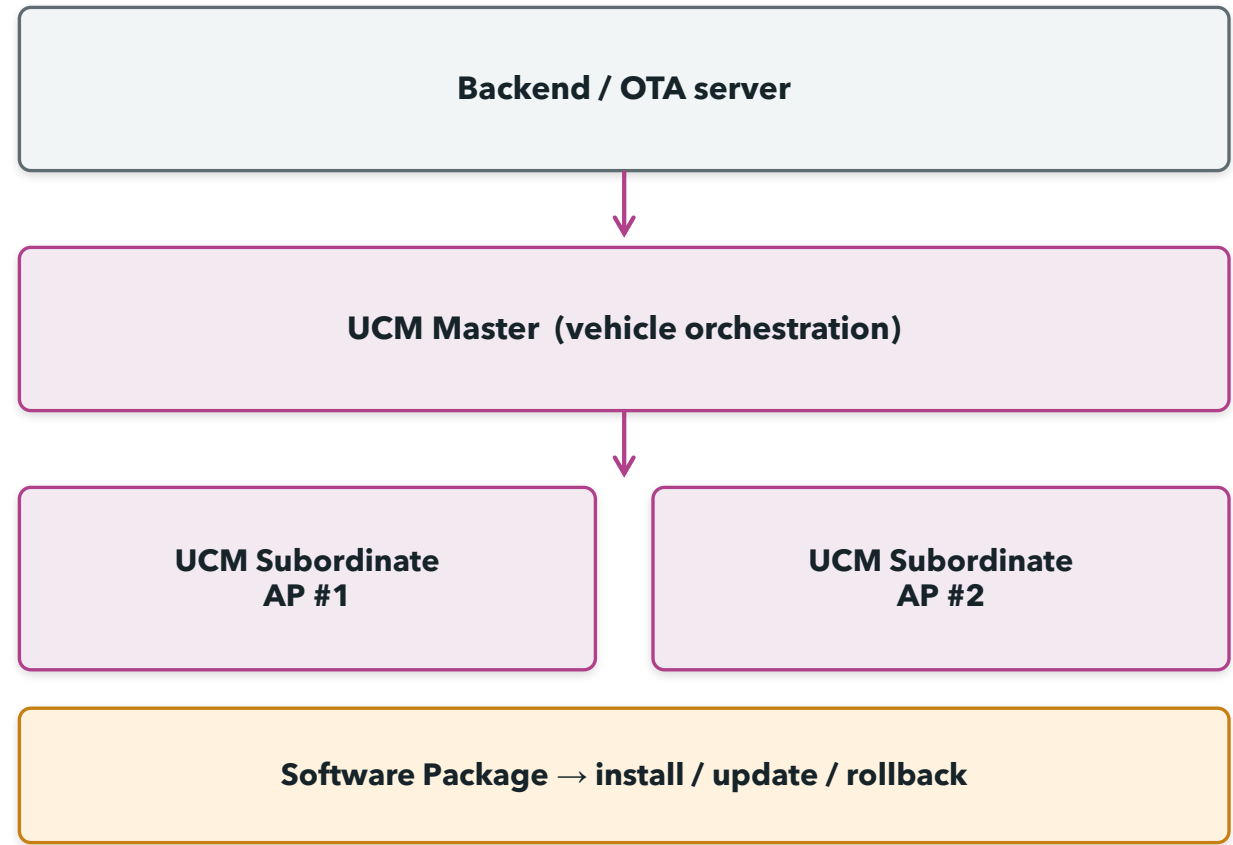
RESPONSIBILITY

Updates, installs, removes and records software on the AP safely and securely – flexibly, over-the-air (OTA).

TWO COMPONENTS

- UCM Subordinate – runs the update on a local AP
- UCM Master – orchestrates updates across the vehicle
- Software Packages carry payload + manifests
- Update campaigns sequence the rollout

SAFE/SECURE Versioned, recorded, and authenticated update flow.



17

Diagnostic Management

DIAGNOSTICS

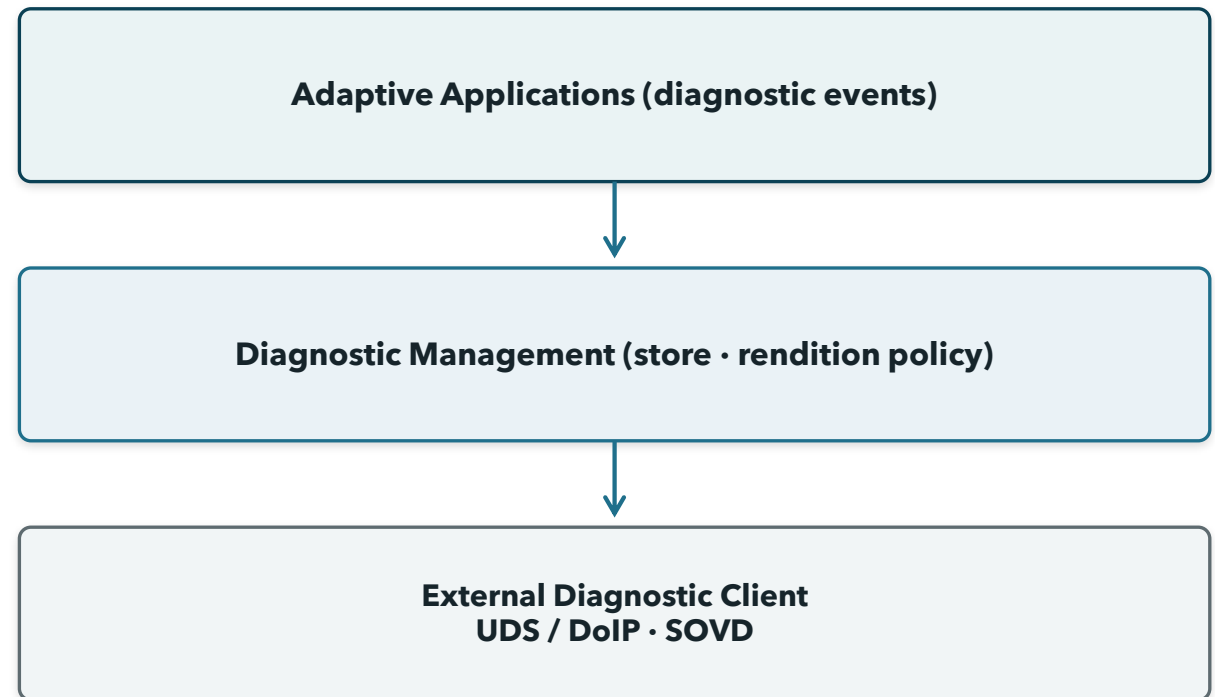
RESPONSIBILITY

Handles diagnostic events from processes in the ARA, stores events + data persistently per rendition policies, and serves external Diagnostic Clients over standard protocols.

STANDARDS

- ISO 14229-1 (UDS) – diagnostic services
- ISO 14229-5 (UDSonIP) – UDS over IP
- ISO 13400-2 (DoIP) – diagnostics over IP
- SOVD – Service-Oriented Vehicle Diagnostics (modern, API/HTTP)

TREND SOVD complements UDS for HPC/connected ECUs.

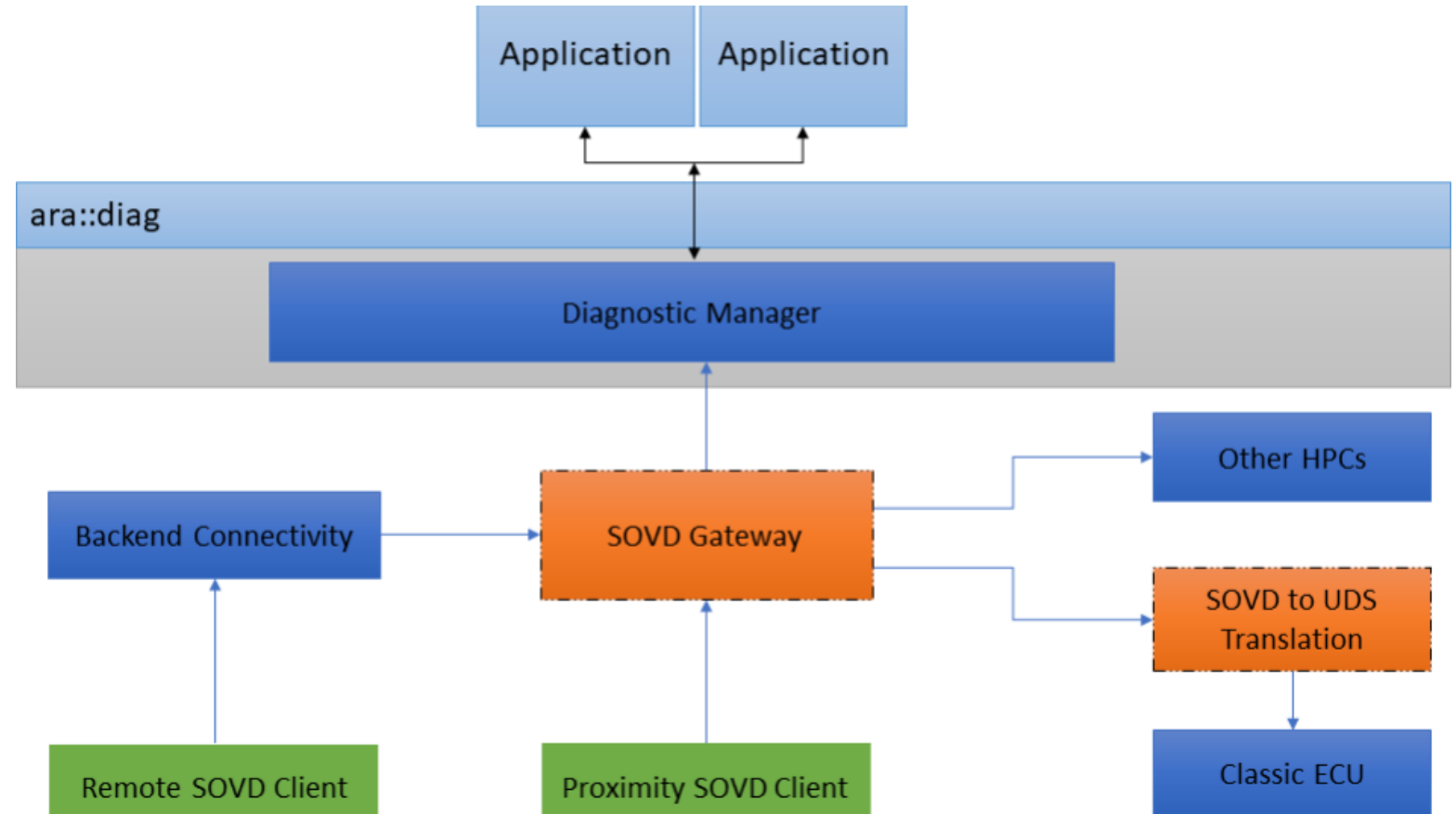


SOVD – Service-Oriented Vehicle Diagnostics

Reference architecture – AUTOSAR Figure 2.1 (SOVD standardized by ASAM)

SOVD PRINCIPLES

- Central edge node for the whole vehicle
- State-of-the-art tech: HTTP / REST / JSON
- Remote, proximity & in-vehicle diagnostics
- Independent of ODX interpretation files
- Encapsulates UDS (ISO 14229-1)
- Designed for HPC use cases



SOVD Reference architecture

SOVD Gateway forwarding is configured via `SOVDGatewayInstantiation`; the SOVD server via `SOVDServerInstance` (TPS_Manifest).

Source: AUTOSAR_EXP_SOVD, R22-11 (SOVD standardized by ASAM)

DIAGNOSTICS · SOVD

REUSE UDS, EXTEND DEXT

Most SOVD use cases map straight onto UDS and reuse the same port instance – no redundant ports. Config reuses the DEXT model, extended only where needed.

DERIVED FROM DEXT

- SOVD data ← DiagnosticDataIdentifier (SOVD_DATA)
- SOVD configuration ← DataIdentifier (SOVD_CONFIGURATION)
- SOVD operation ← DiagnosticRoutine
- SOVD fault ← DiagnosticTroubleCode (DTC)
- Modes ← 0x28 CommunicationControl · 0x85 ControlDTCSetting

DATA INTERPRETATION Diagnostic Manager converts the app byte-stream ↔ SOVD JSON using compuMethods.

SOVD-specific use cases (new interfaces)

Beyond UDS – new Diagnostic Manager interfaces used only by SOVD:

- Authorization – OAuth tokens in the request header; SovdAuthorization resolves the role (reuses Service 0x29 roles)
- Proximity Challenge – SovdProximityChallenge proves client proximity before an operation
- Data-lists – handled natively by the Diagnostic Manager

Why it matters for HPC / connected ECUs

REST/JSON over HTTP makes diagnostics web- and backend-friendly, ODX-independent at the client, and suited to remote fleet workflows.

Example: expose all fault-memory entries of one ECU – or of the entire vehicle – through one SOVD edge node.

Rollout: mandatory features in step 1; optional use cases planned for R23-11.



PART

Methodology & Manifests

Design vs deployment · the three manifests · signal-to-service · software clusters

19

Methodology – design vs deployment

METHODOLOGY

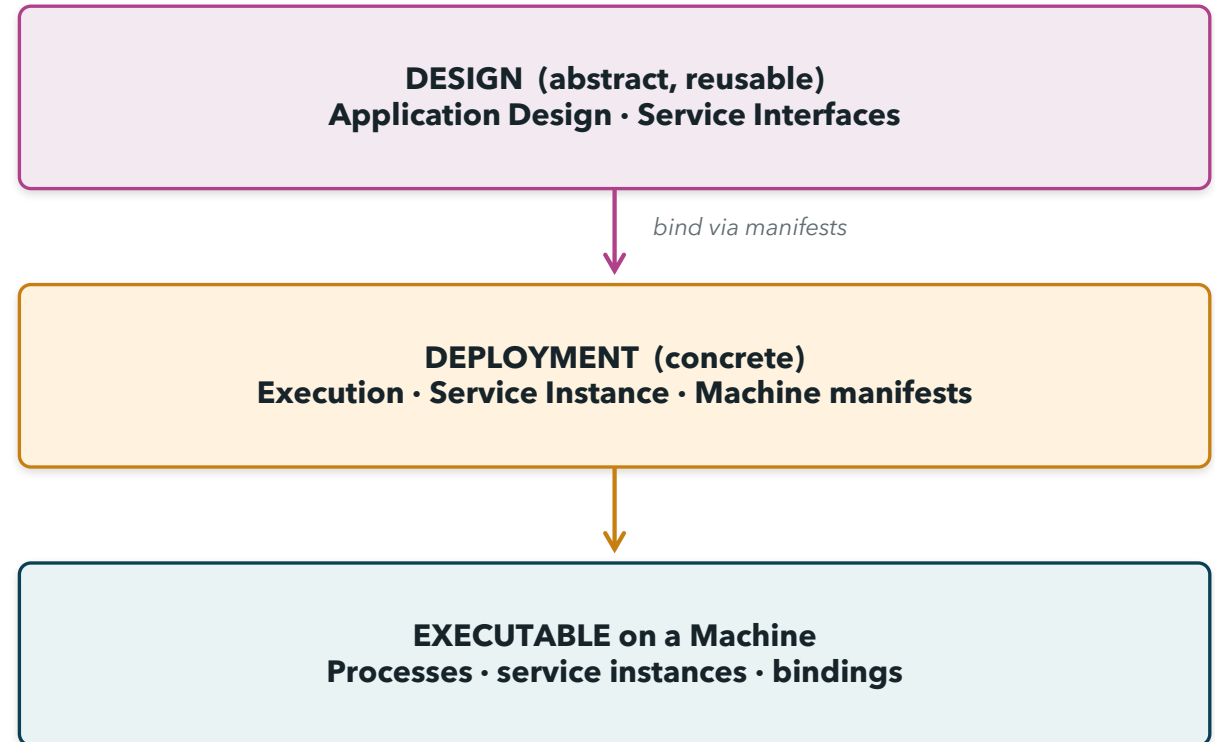
PRINCIPLE

AP development standardizes work products describing services, applications, machines and their configuration. Application design is kept independent of any specific deployment.

WORK PRODUCTS

- System Design – software components & service interfaces
- Service Interface Design – the pivotal SOA element
- Application Design – abstract, deployment-independent
- Machine Design – the target AP instance
- Software Cluster Design – packaging & mapping

GOAL Reuse application software across different deployment scenarios.



METHODOLOGY

Execution Manifest

Deployment-related info to instantiate an application:

- Controls application instantiation
- Bounds dynamic behaviour (planned dynamics)
- Bundled with the executable

per executable

Service Instance Manifest

How service-oriented comm is configured on the network:

- Maps services to transport (e.g. SOME/IP)
- Provided vs required instances
- Bundled with the executable

per service usage

Machine Manifest

Deployment info for the Machine itself:

- Machine / OS configuration
- Network endpoints
- Establishes the AP instance

per machine

Application Design – abstract artifacts (service interfaces, access definitions, app grouping) that AID writing the Execution & Service Instance manifests, independent of deployment

21

Signal-to-Service translation

METHODOLOGY

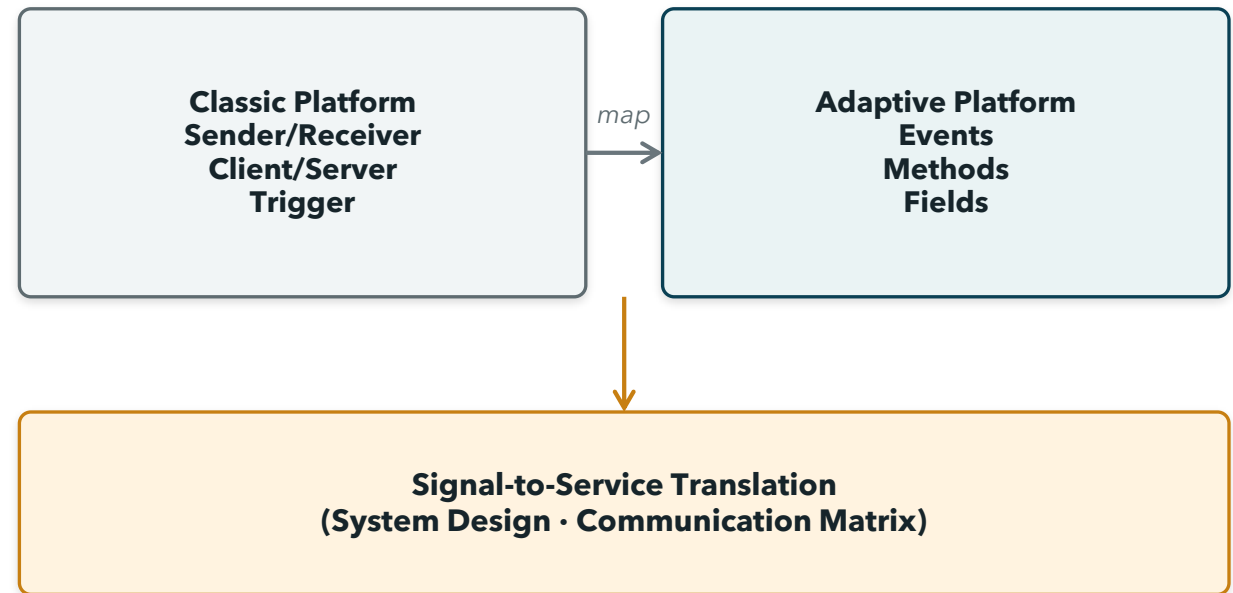
WHY

Bridges service-oriented AP comms with signal-based Classic Platform comms – used when the CP side does not already fulfill the SOME/IP message format.

MAPPING

- Map events ↔ Sender/Receiver
- Map methods ↔ Client/Server
- Map fields ↔ signals
- Map to SOME/IP messages
- Driven by the CP Communication Matrix + System Design

RESULT AP and CP applications interoperate over SOME/IP.



METHODOLOGY

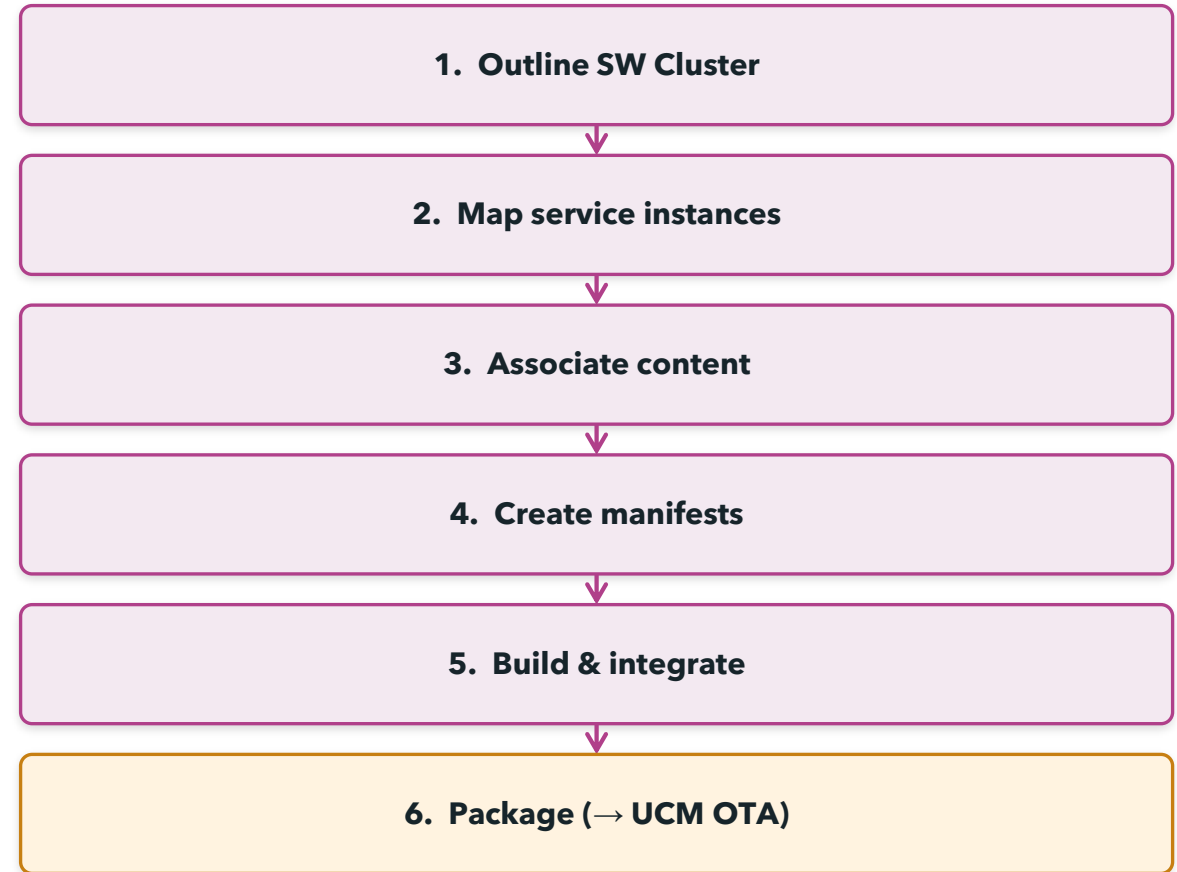
SOFTWARE CLUSTER

A deployable unit grouping content (executables, manifests, service instances) for build, integration and update.

STEPS

- Outline SW Cluster design
- Define & map provided/required service instances
- Associate content elements
- Build for test env → create Execution Manifest
- Finalize Service Instance Manifest · integrate diagnostics

THEN Package as a Software Package; specify the update campaign (→ UCM).





PART

Design Principles

C++14 guidelines · ara::core error handling · SOLID + Acyclic Dependencies

23

C++14 guidelines & error handling

PRINCIPLES

RESERVED NAMESPACE

The namespace `ara` (and everything below it) is reserved by AUTOSAR – applications must not add definitions there.

NO EXCEPTIONS FOR ASIL

- ASIL-certified C++ toolchains often lack exception support
- AP defines error handling WITHOUT C++ exceptions
- `ara::core::ErrorCode` – enum value + error domain
- `ara::core::Result<T,E>` – value or error, checked
- `ara::core::Future / Promise` for async results

RULE Coding guidelines make a complex language safe & secure.

```
// ara::core error handling (no exceptions)
ara::core::Result<Speed> ReadSpeed() noexcept;

auto r = ReadSpeed();
if (r.HasValue()) {
    use(r.Value());
} else {
    ara::core::ErrorCode e = r.Error();
    log(e);                // enum + error domain
}

// async: Future / Promise
ara::core::Future<Cfg> f = proxy.Calibrate(c);
```

24

SOLID & Acyclic Dependencies

PRINCIPLES

Single Responsibility

Where a component owns one part of functionality
Roles minimizes reasons to change an interface → maintainable

Interface Segregation

Where clients don't depend on methods they don't use
Roles split interfaces by client role

Dependency Inversion

Where depend on abstractions, not implementations
Roles decouples FCs & layers

Acyclic Dependencies Principle

Where the component dependency graph has no cycles
Roles enables independent build, test & release of clusters

Leverage existing standards

Where reuse C++, POSIX, SOME/IP – don't reinvent
Roles no casual new interface where a standard exists



PART

Validation — System-Test Use Cases

Concrete scenarios that confirm each functional cluster (AUTOSAR R22-11 System Tests)

How the use cases are tested

ISO 9646 test architecture on the AP Demonstrator (Raspberry Pi 3B / 4)

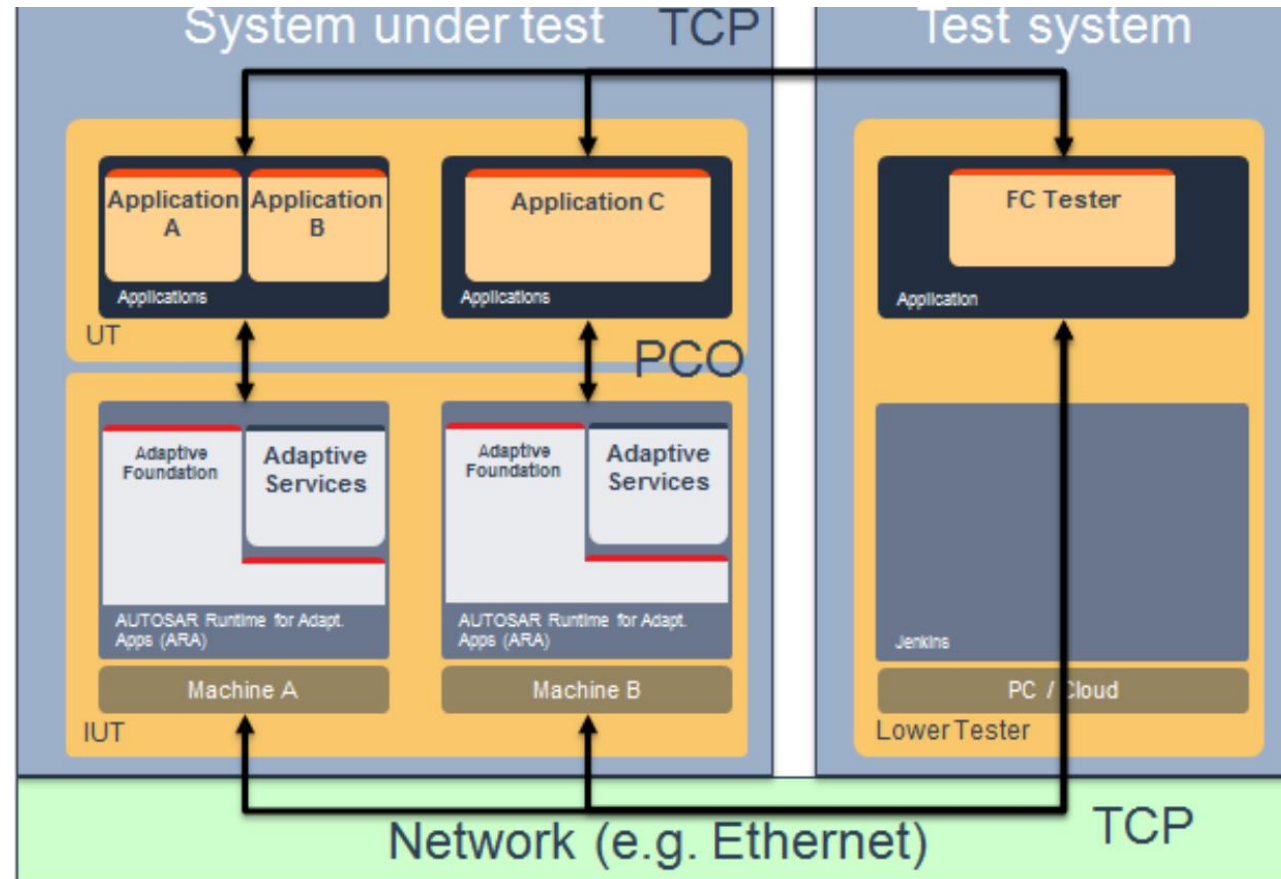


Figure 2.1 — System Test architecture (LT / IUT / UT · PCO · TCP)

What System Tests are

Validate RS items – do the functional-cluster requirements hold on the AP Demonstrator?

- In scope: CM (SOME/IP), EMO, DIAG, LT, PER, IAM, UCM, E2E, TS, SEC, NM, CRYPTO
- PCO = Point of Control & Observation via ARA::API + Ethernet
- TCP = Test Coordination via a Diagnostic routine service

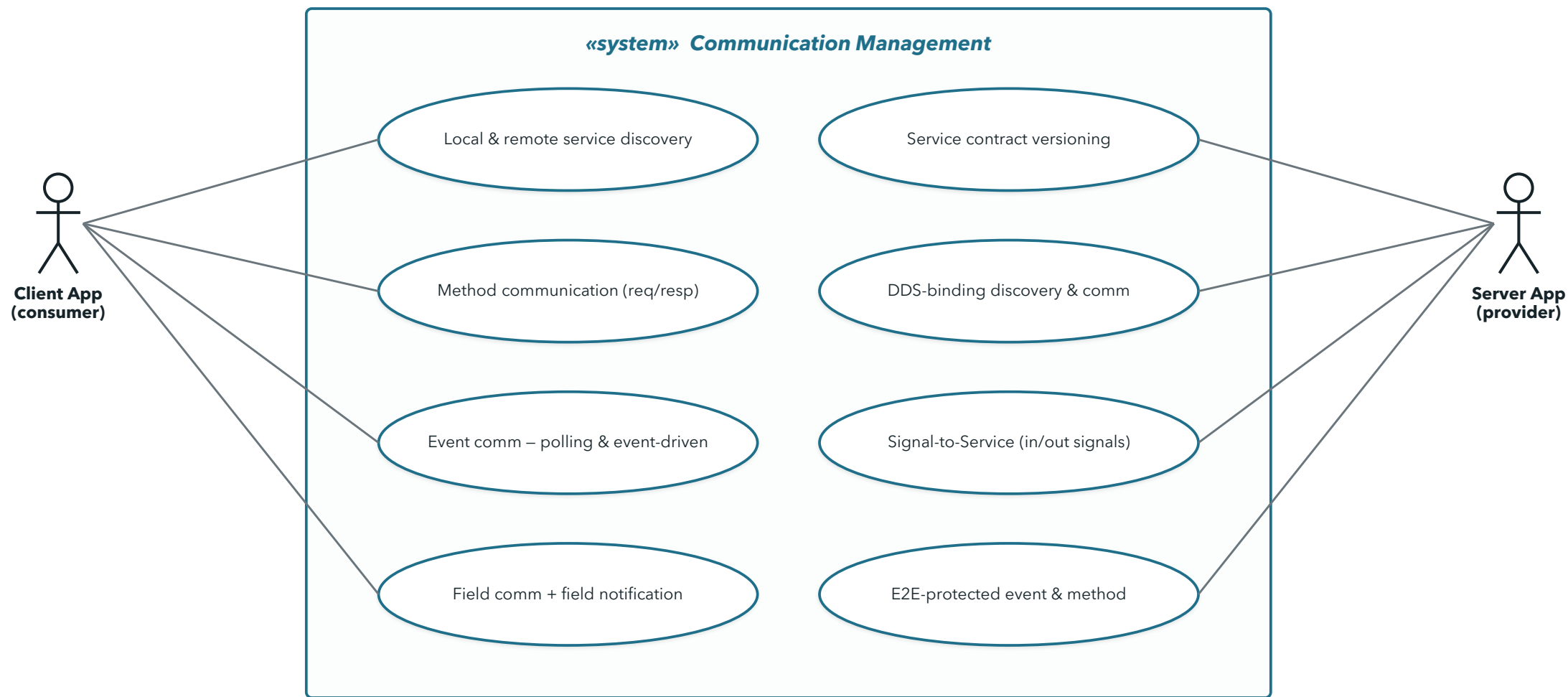
Read these as use cases

Each [STS_*] case is an end-to-end scenario, not a unit test.

- "System test cases are just examples – there are many ways to define and implement use-case scenarios"
- Setups are reference only; may cover broader scope
- The following slides group these scenarios by cluster

Communication Management – use cases

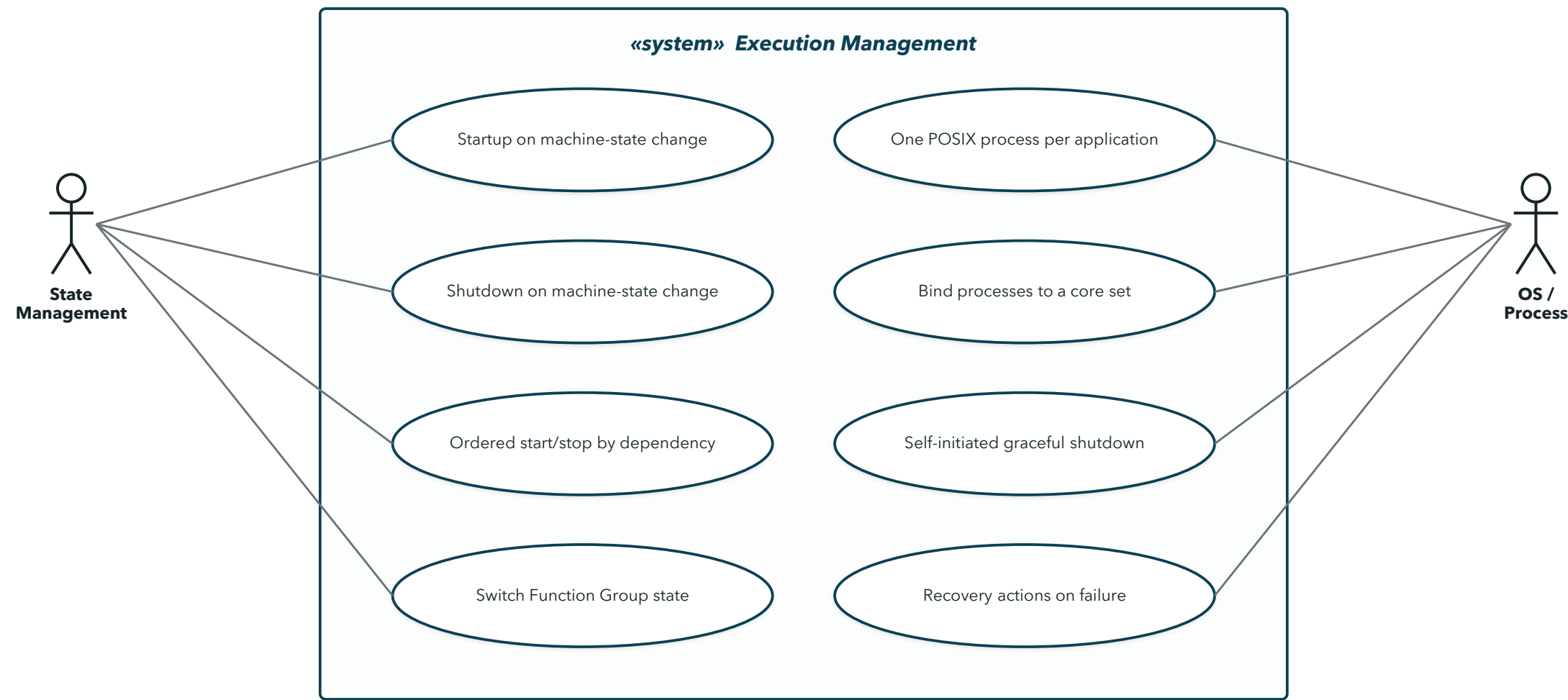
[STS_CM_00001..10] · [STS_DDS_00001..04] · Signal-to-Service



Covers all four ara::com mechanisms over SOME/IP and DDS bindings, plus CP↔AP signal bridging and end-to-end protection.

Execution Management – use cases

[STS_EMO_00001..14]



EM owns the process lifecycle: only EM starts Processes; it enforces dependencies, core binding, resource limits and recovery.

Diagnostics – UDS service use cases

[STS_DIAG_00001..17] over DoIP (ISO 14229 / ISO 13400)

0x22 ReadDataByIdentifier

0x31 RoutineControl

0x2F InputOutputControl

0x19 ReadDTCInformation

0x85 ControlDTCSetting

0x2E WriteDataByIdentifier

0x3E TesterPresent

0x14 ClearDTC

0x27 SecurityAccess

0x34 RequestDownload

Beyond raw services

DTC handling:

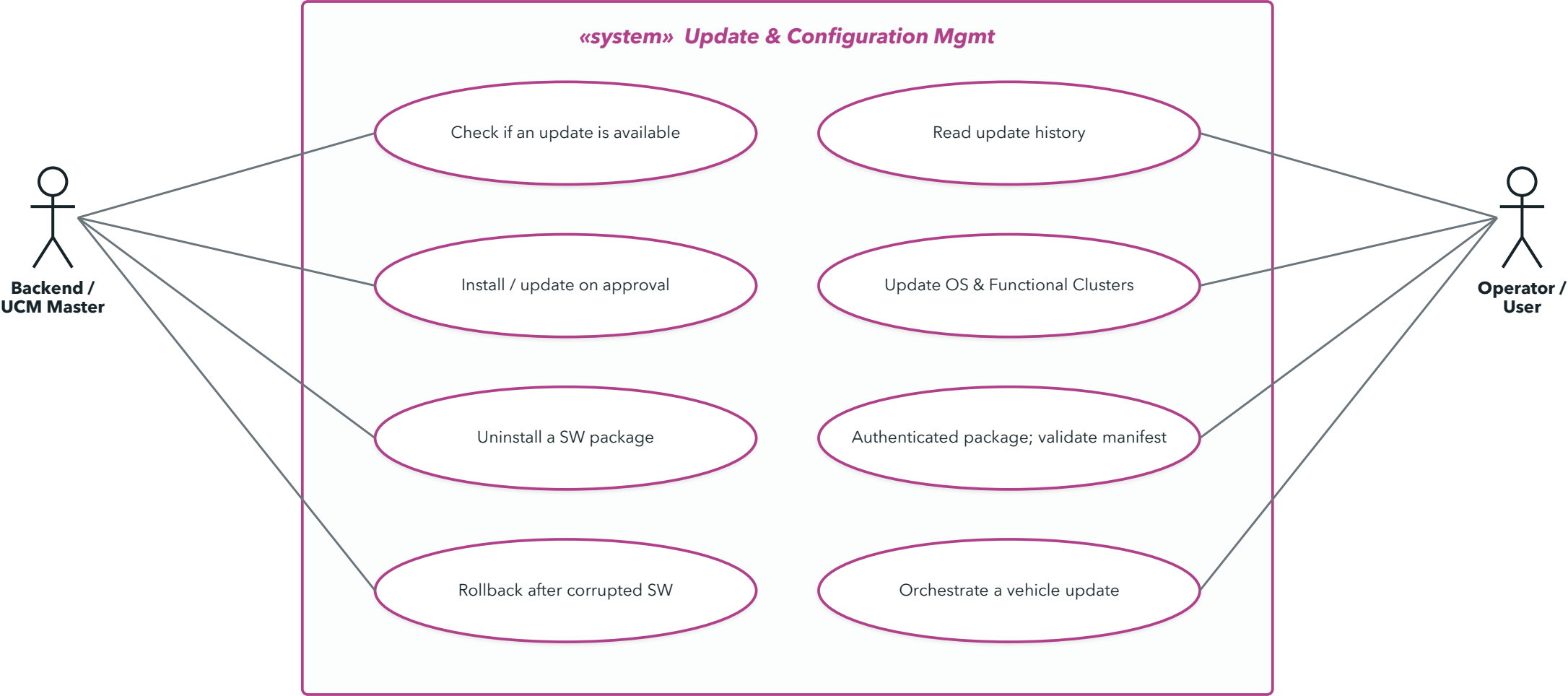
- Store & read DTC status + snapshot data
- Aging of confirmedDTC / testFailedSinceLastClear
- Configurable event debounce counter
- Freeze debounce when ControlDTCSetting=Disabled

Flow control:

- Pending response (0x78) on excess payload
- RequestDownload for SW transfer
- Connection-specific meta-info to ext. processors

Update & Configuration Management — use cases

[STS_UCM_00001..15]



Full OTA lifecycle: multi-client transfer, cancel, multi-package install, and vehicle-level orchestration via UCM Master.

Security & data-integrity use cases

Cryptography · E2E · IAM · Security Management

Cryptography [STS_CRYPTO_00001..18]

Where enc/dec (sym & asym), MAC, digital signature, hash, RNG, AEAD, key wrap/encapsulation, key-usage restriction, Diffie-Hellman exchange, key import/export, PKI/X.509

Roles ara::crypto · HSM-backed

E2E Protection [STS_E2E_00001..03]

Where AP→AP protection for Event and Method communication; detect a corrupting application affecting the data stream

Roles checks common E2E profile parts

Identity & Access Mgmt [STS_IAM_00001..04]

Where reject local service use by unauthorized app; reject events from/with no authorized party; app-provided access-control decision

Roles deny-by-default access control

Security Management [STS_SEC_00001..02]

Where message authentication; message confidentiality and integrity end-to-end

Roles SecOC-style protection

Platform-service use cases

Persistence · Health · State · Time Sync · Network · Logging

Persistence [STS_PER_00001..08]

Store/retrieve int·float·string in key-value DB & files; survive reboot; storage limits; encrypted file

Platform Health [STS_HM_00001..03]

Alive supervision · Deadline supervision · Logical supervision

State Management [STS_SM_00001]

Coordinate & control Function Group states across the machine

Time Sync [STS_TS_00001..05]

Offset master/slave TimeBase APIs; sync of applications across ECUs

Network Mgmt [STS_NM_00001..02]

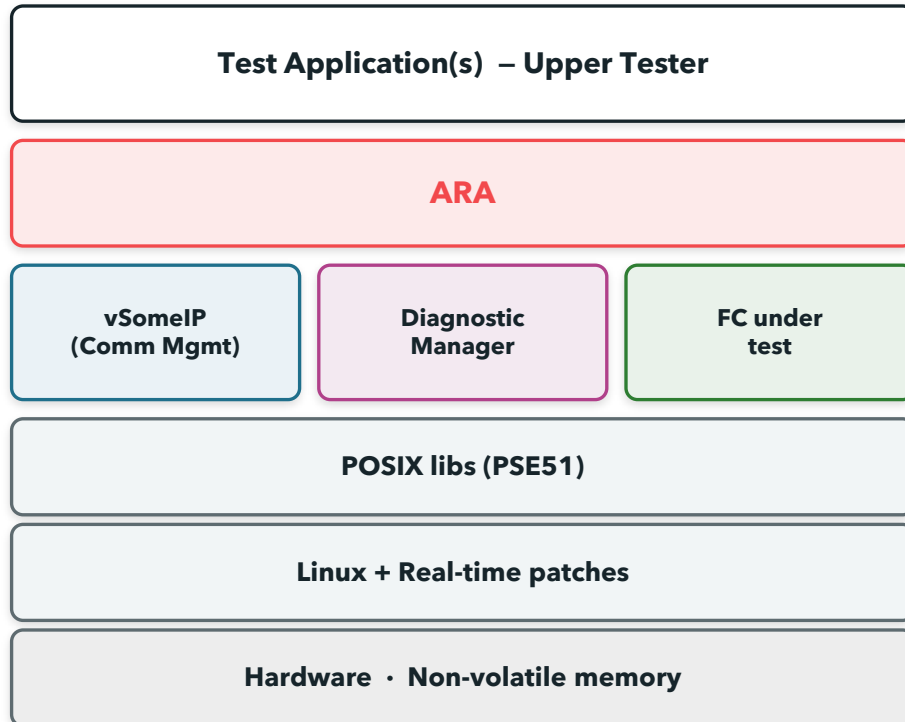
Basic NM for ECUs in / not in the coordinated network

Logging & Tracing [STS_LT]

Per-module log levels; collect logs across several ECUs

Reading the test setups – the recurring FC pattern

Every figure stacks the same way; the boxes reveal which Functional Clusters each use case pulls in



Lower Tester = Jenkins (LT) → drives & observes over Ethernet

Comm Mgmt = observation (PCO)

vSomelP appears in almost every setup: the Point of Control & Observation runs over ARA::API + Ethernet (SOME/IP). Discovery/events/methods are how the tester sees IUT behaviour.

Diagnostics = coordination (TCP)

The Diagnostic Manager box recurs because Test Coordination is realized by stimulating apps via a Diagnostic routine service (RoutineControl) – so DIAG co-participates in EM, PER, UCM, TS, Crypto...

Cluster-specific element

The third box is the FC under test: Execution Manager, Time Sync, Cryptography, NM, Persistency (NV memory), etc. Local-only clusters (PHM, PER) drop vSomelP.

Functional Clusters used per use case

Derived from the 'Illustration of test setup' figures (Sections 4-17)

Use case (FC under test)	Comm Mgmt	Diag	Exec Mgmt	State Mgmt	Cluster-specific element / transport
Communication Mgmt [CM/DDS/S2S]	●				ara::com · SOME/IP · DDS
Execution Mgmt [EMO]		○	●	○	Exec Manager · Machine State Client
Diagnostic Mgmt [DIAG]	○	●			Diagnostic Manager · DoIP
Log and Trace [LT]					DLT over TCP (:3490)
Persistency [PER]		○			Key-Value / File · NV memory
Identity & Access Mgmt [IAM]	○				access control on services
Update & Config Mgmt [UCM]	○	○			SW packages · OTA
E2E Protection [E2E]	●	○			realized within Comm Mgmt
Time Synchronization [TS]	○	○			Time Sync module
Security Mgmt [SEC]	○				SecOC · msg auth/confidentiality
Network Management [NM]	○				NM module (3 ECUs)
Cryptography [CRYPTO]		○			Crypto stack · HSM
Platform Health Mgmt [HM]					Watchdog · NV memory (local)
State Management [SM]		○	○	●	Exec Manager · Machine State Client

● FC under test ○ supporting FC shown in the test-setup figure · nearly every setup uses Comm Mgmt (observation) + Diagnostics (coordination)

Source: AUTOSAR_TR_AdaptivePlatformSystemTests, R22-11

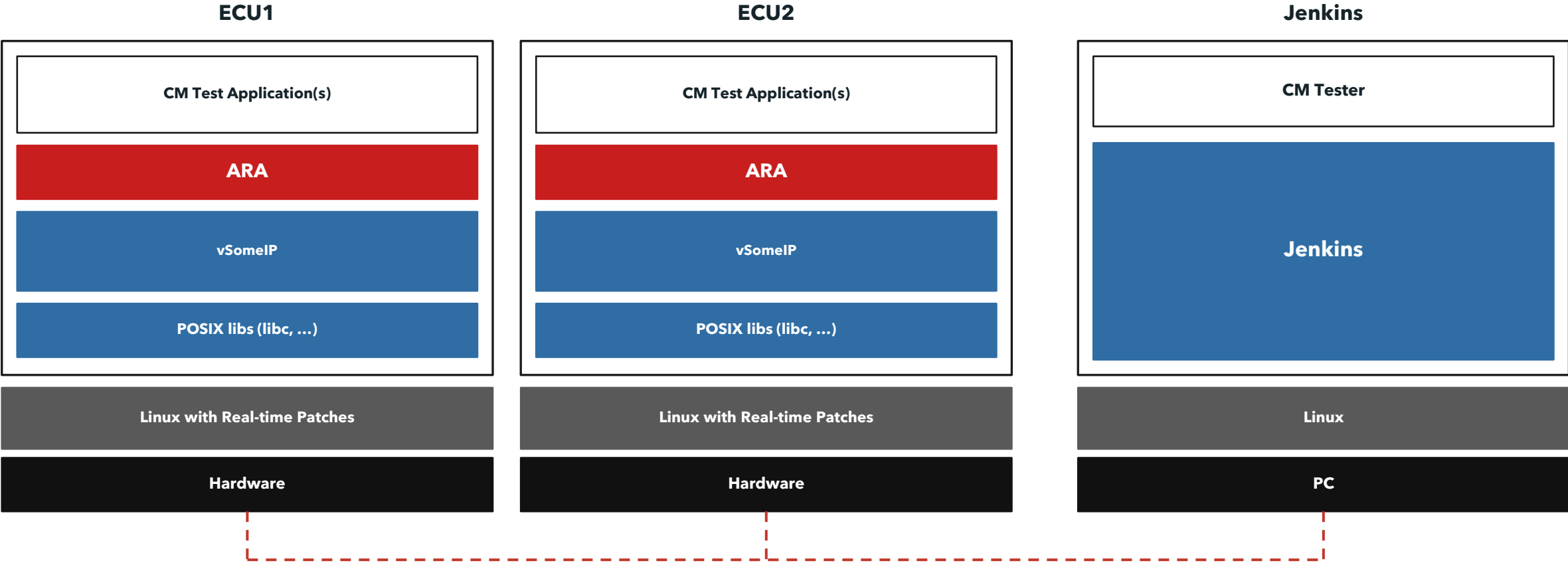
PART

Test-Setup Diagrams

Recreated from the 'Illustration of test setup' figures – involved ECUs + Jenkins, with the real FC boxes

Communication Management – test setup

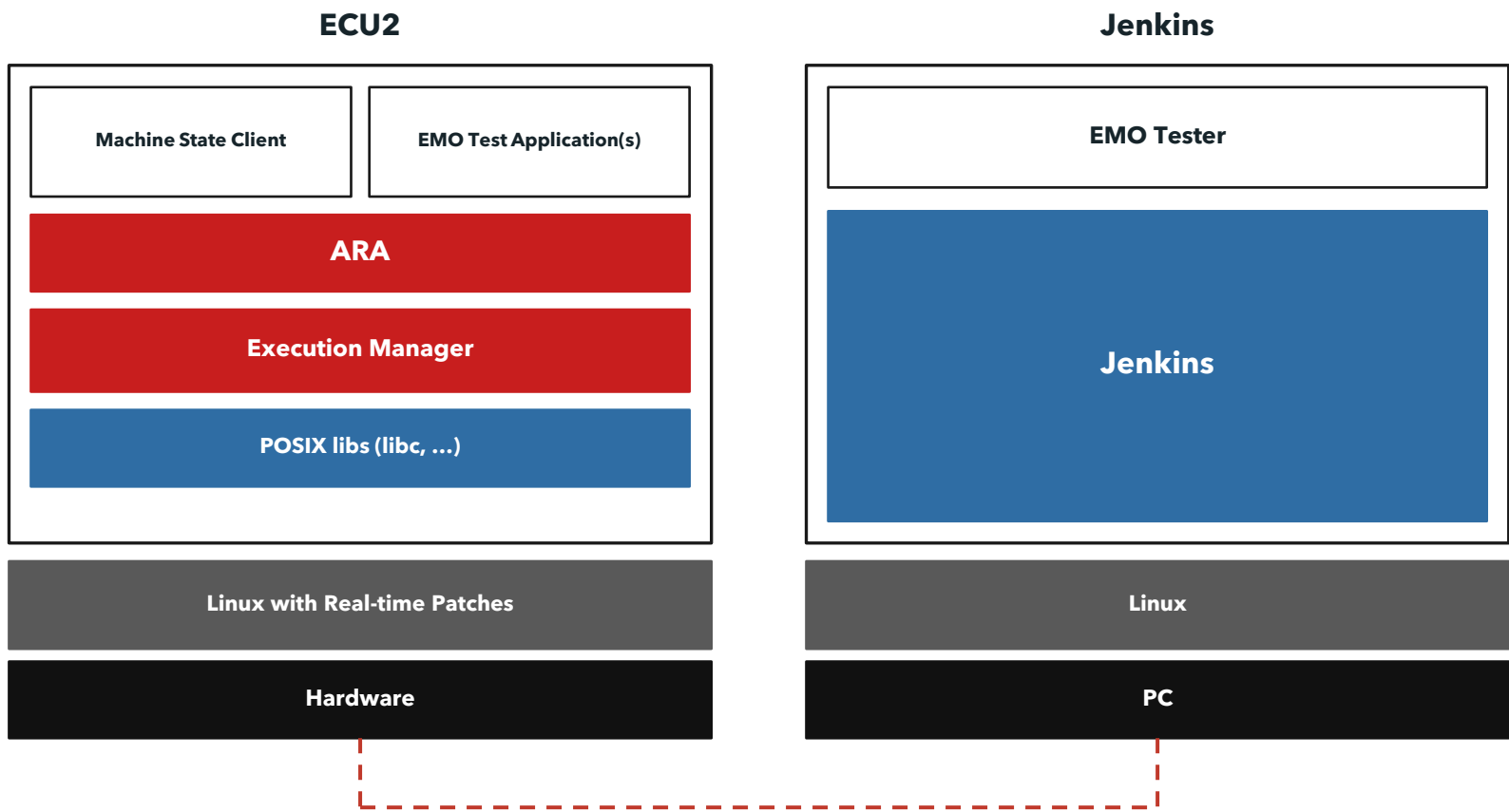
Figure 4.1 · [STS_CM] · [STS_DDS] · Signal-to-Service



Functional clusters involved: Communication Management (ara::com · SOME/IP · DDS)

Execution Management — test setup

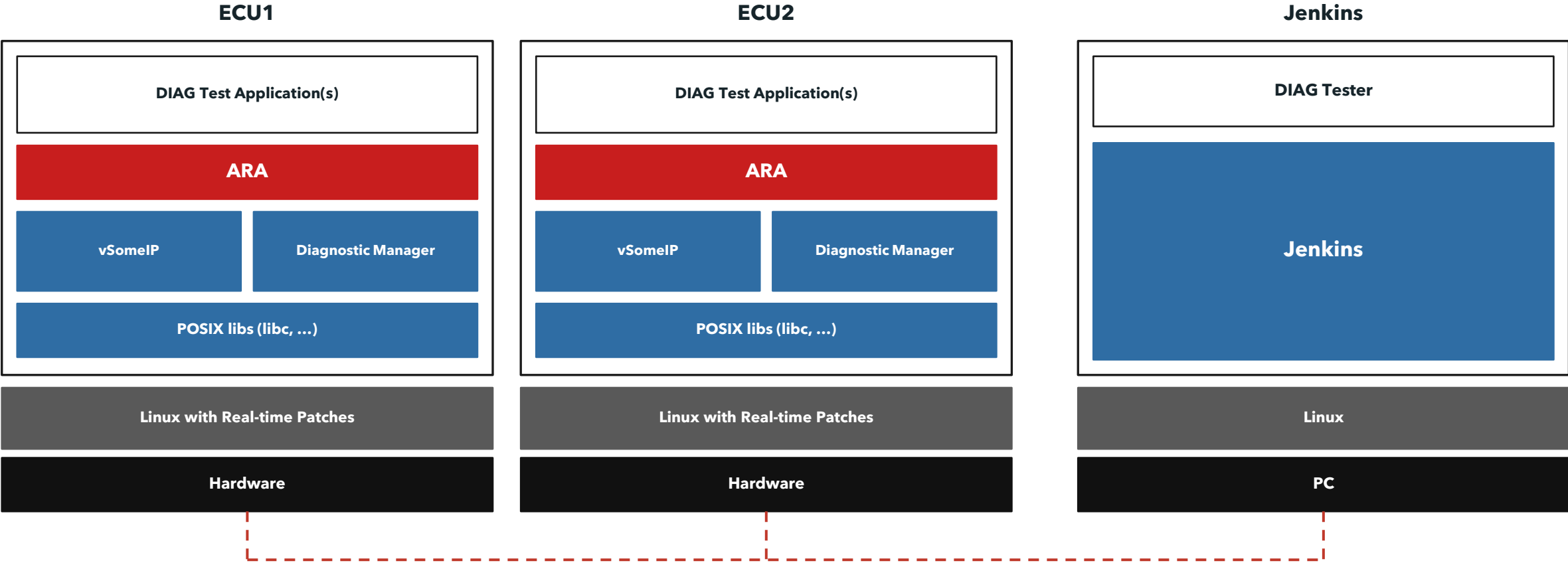
Figure 5.1 · [STS_EMO]



Functional clusters involved: Execution Management + State Management (Machine State Client); test coordination via Diagnostics

Diagnostics — test setup

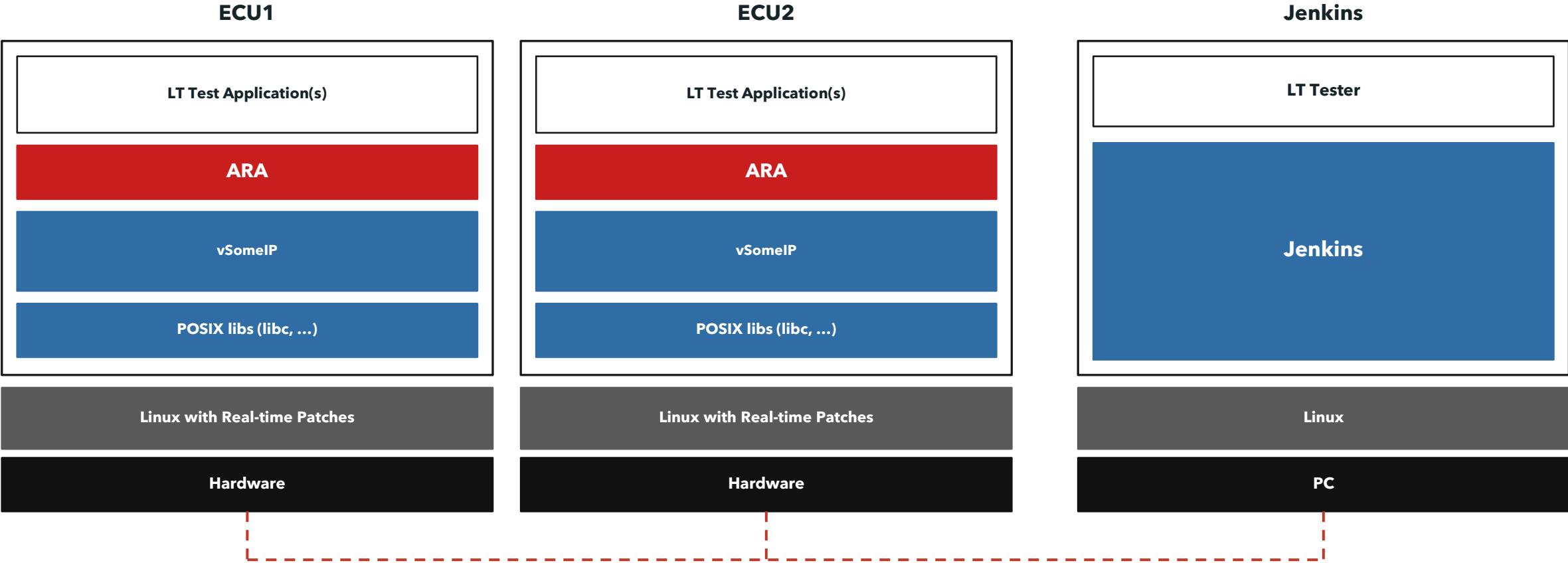
Figure 6.2 · [STS_DIAG]



Functional clusters involved: Diagnostic Management + Communication Management (vSomelP)

Logging & Tracing – test setup

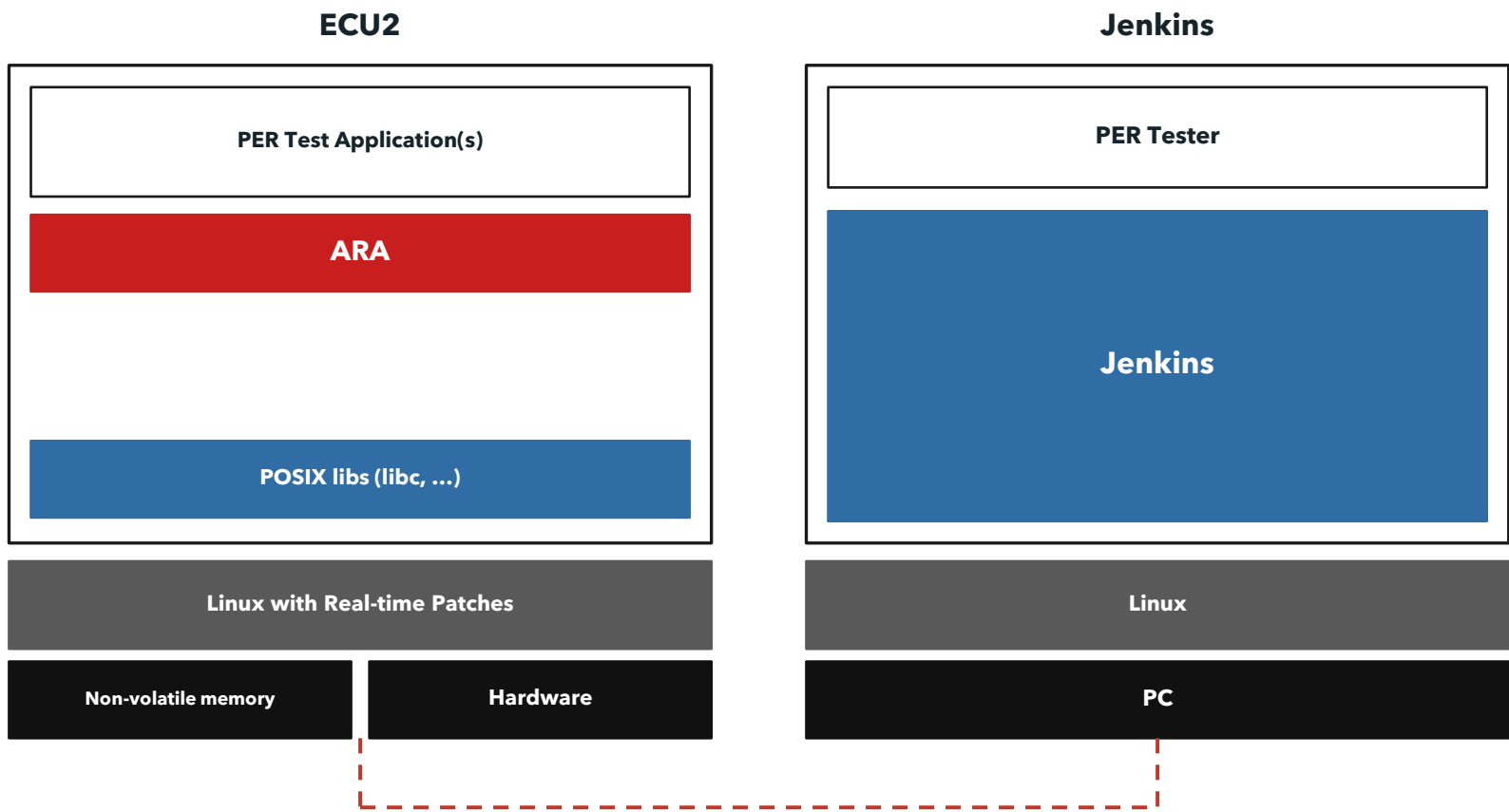
Figure 7.1 · [STS_LT]



Functional clusters involved: Log and Trace (DLT over TCP :3490) + Communication Management

Persistency — test setup

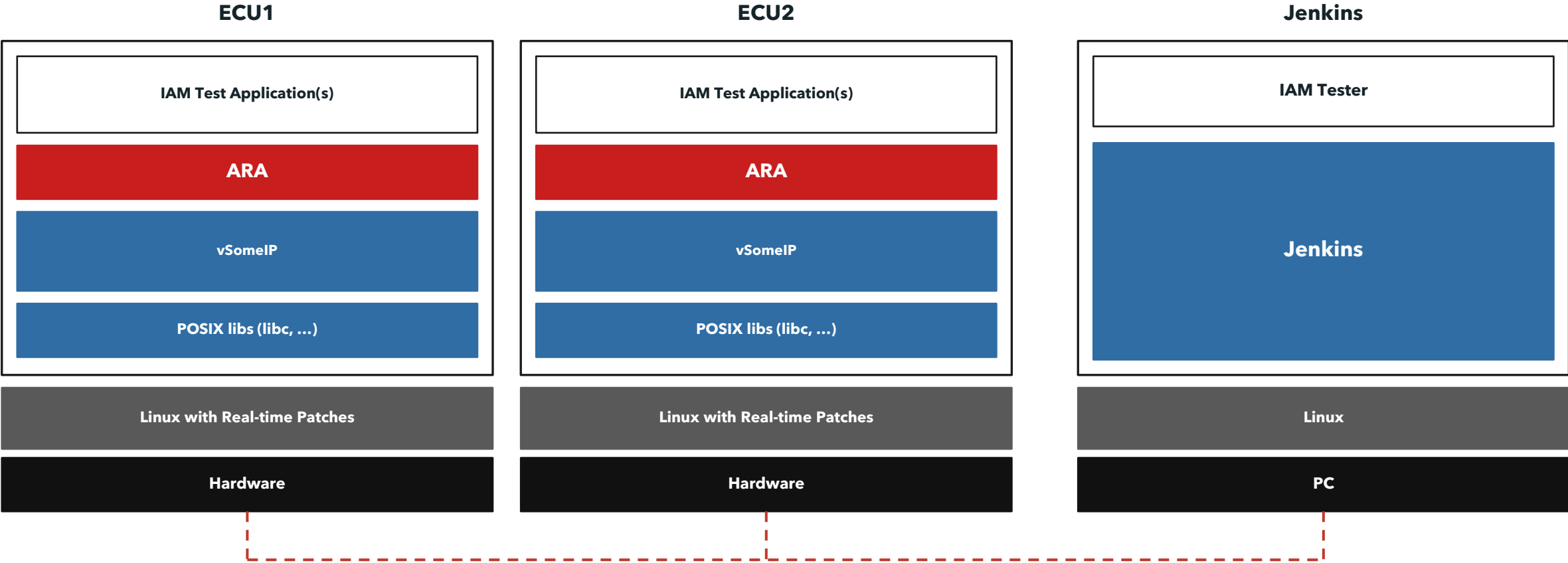
Figure 8.1 · [STS_PER]



Functional clusters involved: Persistency (NV memory); coordination via Diagnostics (RoutineControl)

Identity & Access Management – test setup

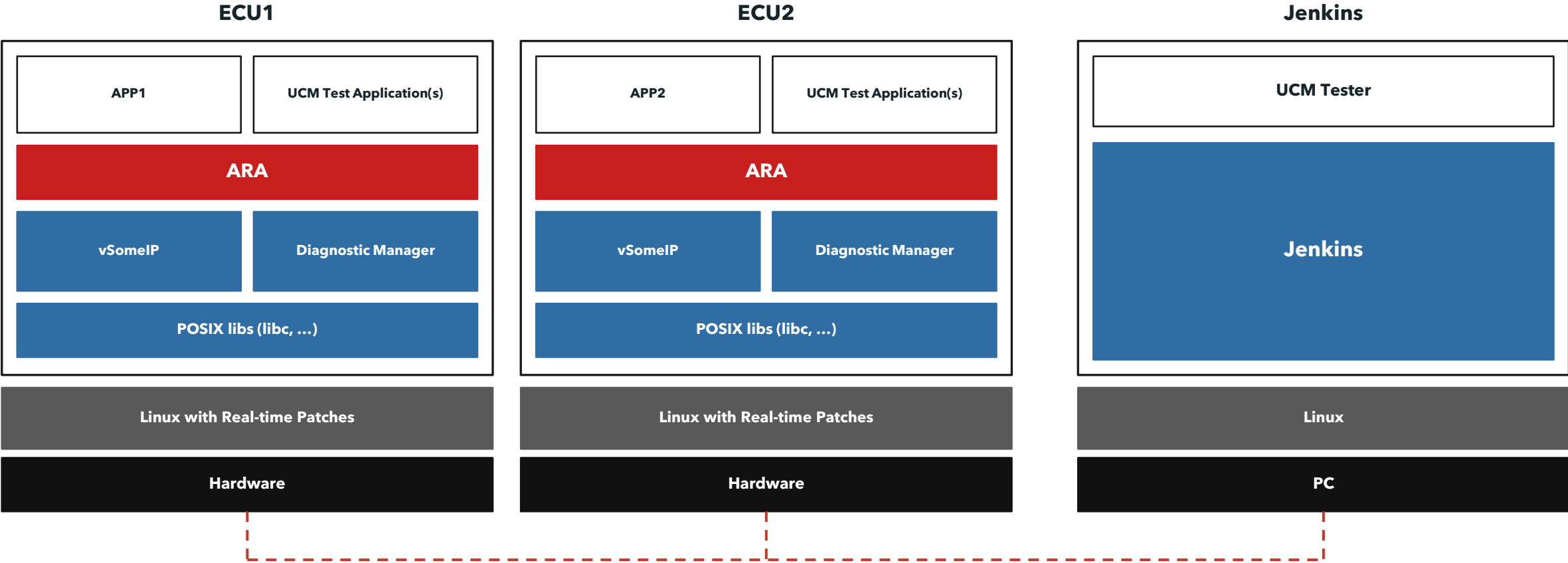
Figure 9.1 · [STS_IAM]



Functional clusters involved: Identity & Access Management + Communication Management (vSomeIP)

Update & Configuration Management — test setup

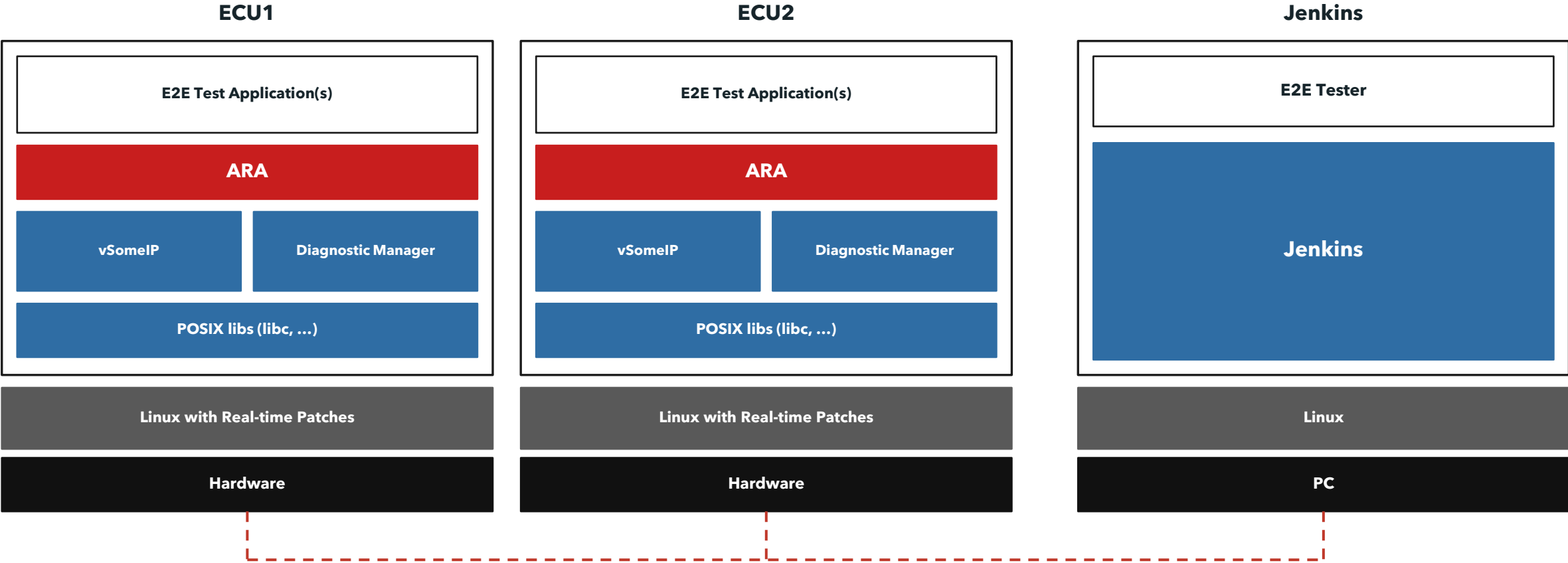
Figure 10.1 · [STS_UCM]



Functional clusters involved: UCM + Communication Management (vSomeIP) + Diagnostics (Diagnostic Manager)

E2E Protection — test setup

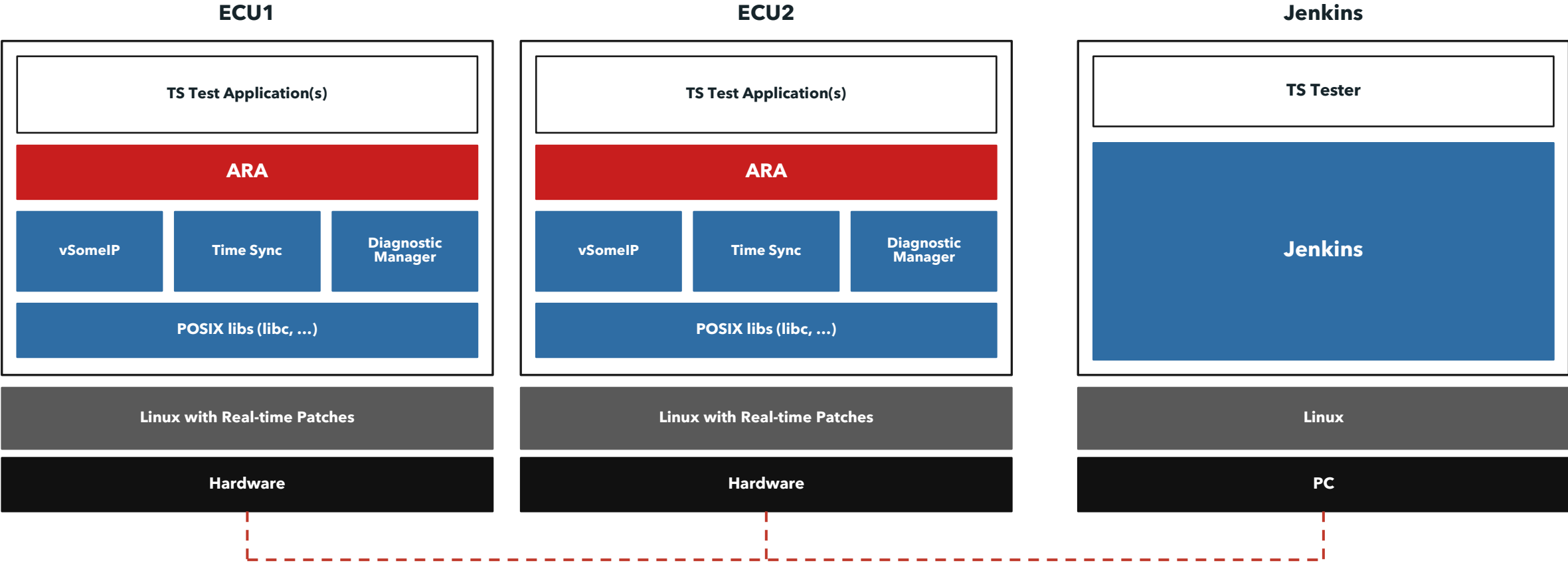
Figure 11.1 · [STS_E2E]



Functional clusters involved: E2E (within Communication Management) + Communication Management (vSomelP) + Diagnostics

Time Synchronization — test setup

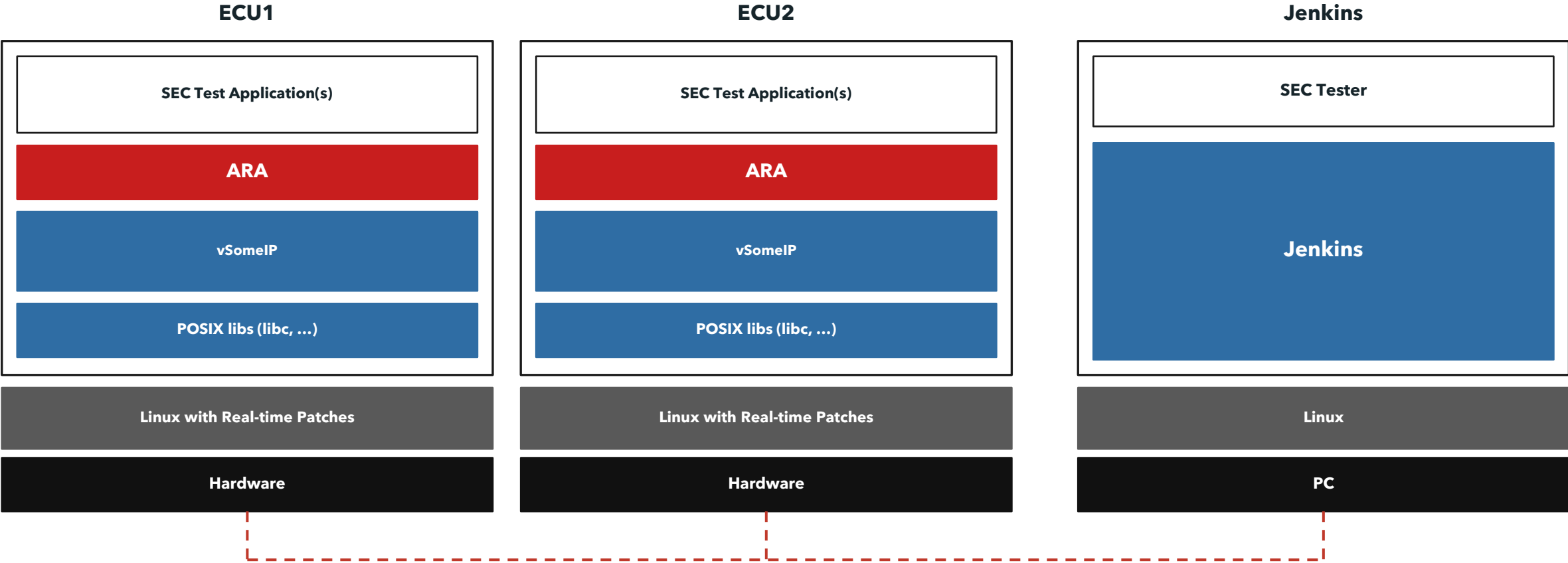
Figure 12.1 · [STS_TS]



Functional clusters involved: Time Synchronization + Communication Management (vSomeIP) + Diagnostics

Security Management — test setup

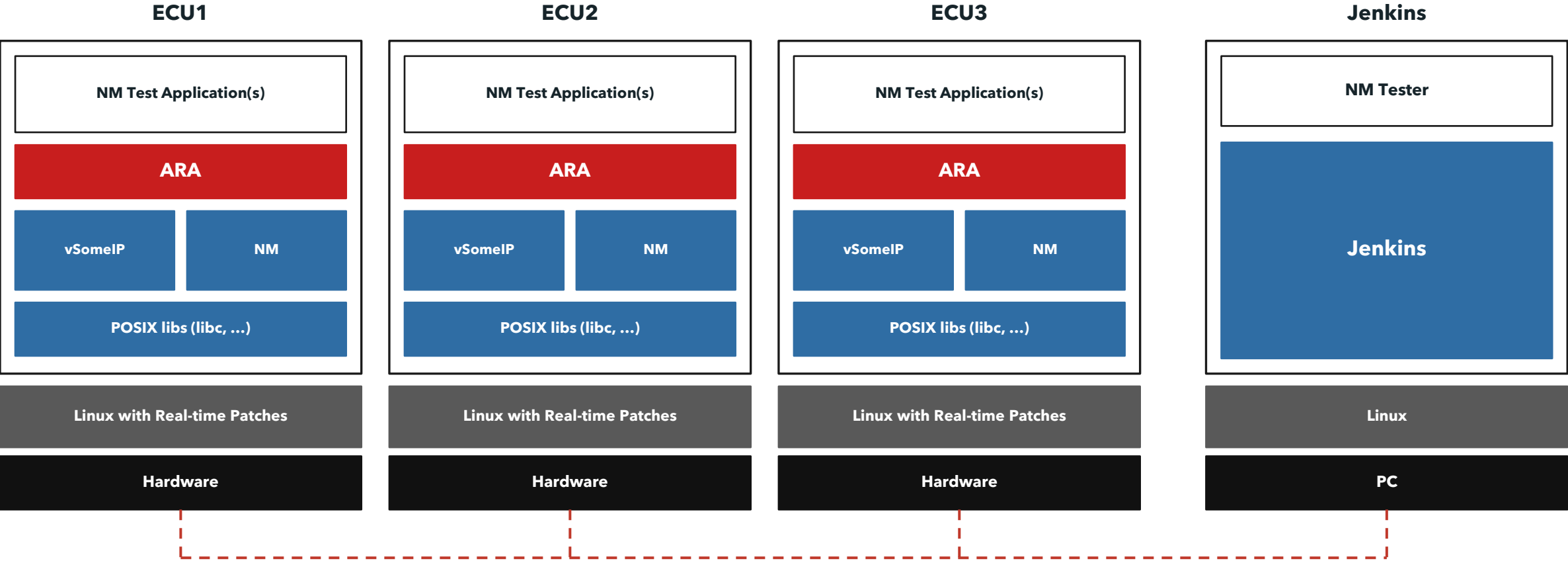
Figure 13.1 · [STS_SEC]



Functional clusters involved: Security (SecOC) + Communication Management (vSomeIP)

Network Management — test setup

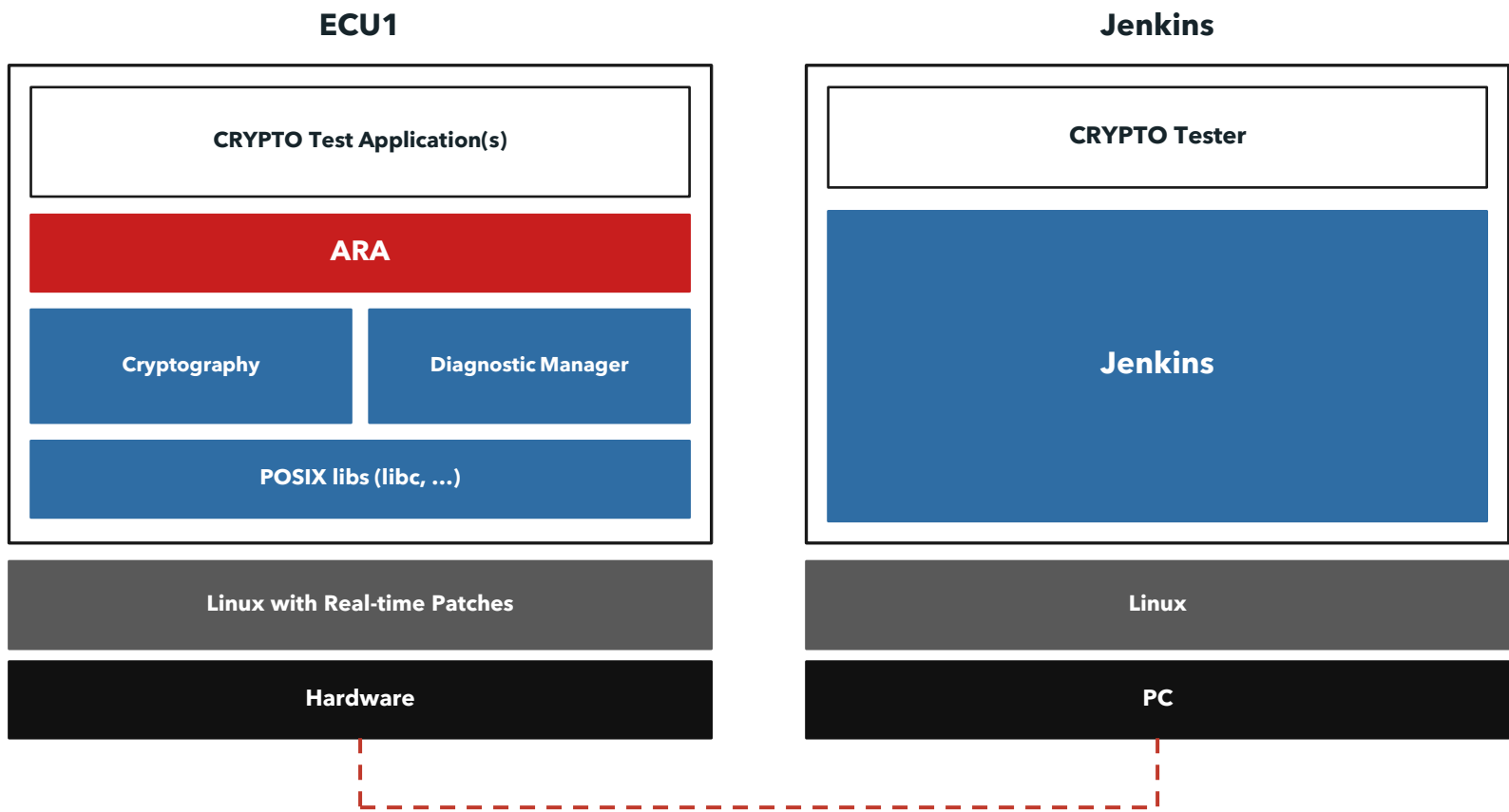
Figure 14.1 · [STS_NM]



Functional clusters involved: Network Management + Communication Management (vSomeIP)

Cryptography – test setup

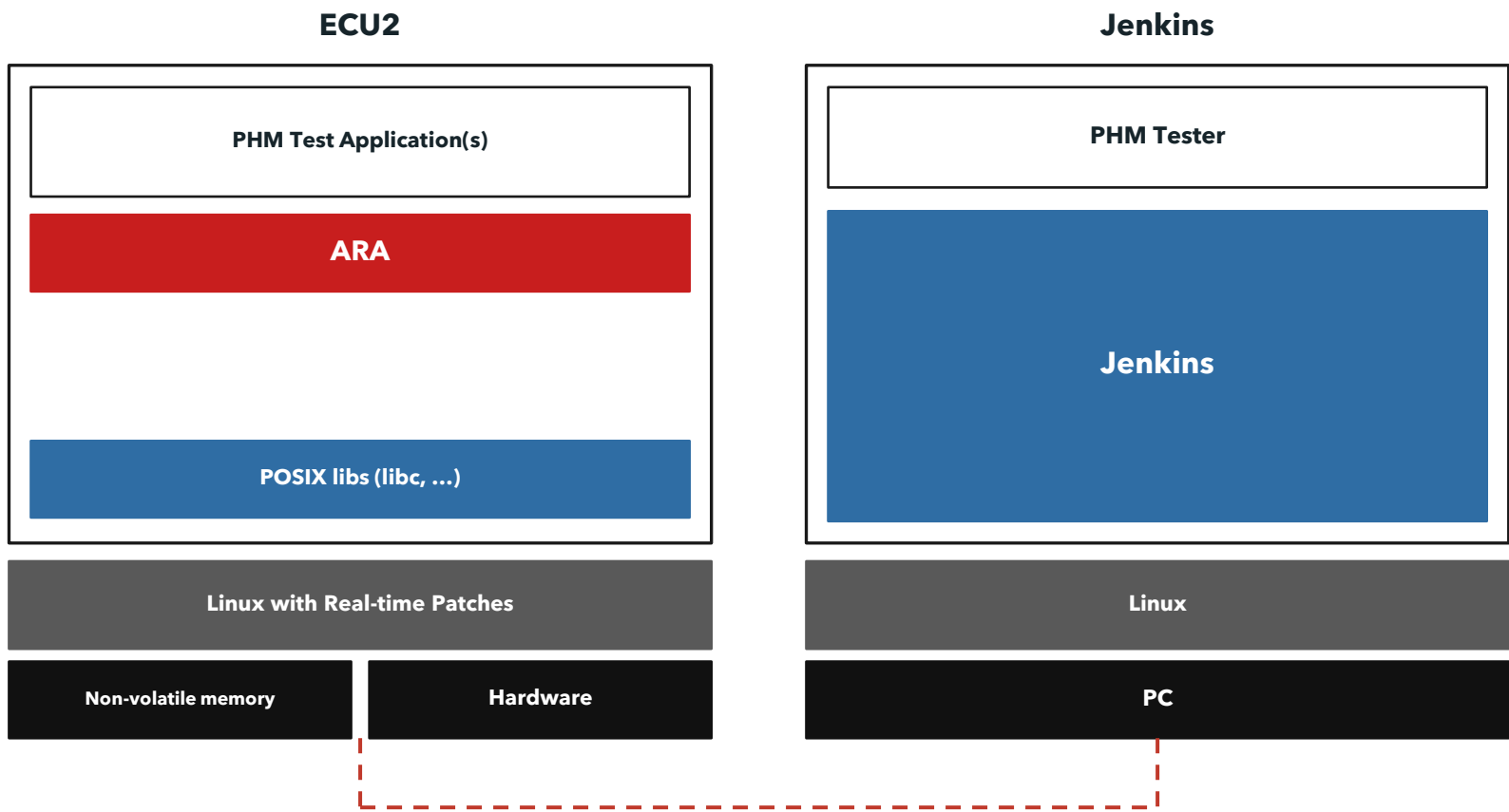
Figure 15.1 · [STS_CRYPT0]



Functional clusters involved: Cryptography + Diagnostics (test coordination)

Platform Health Management — test setup

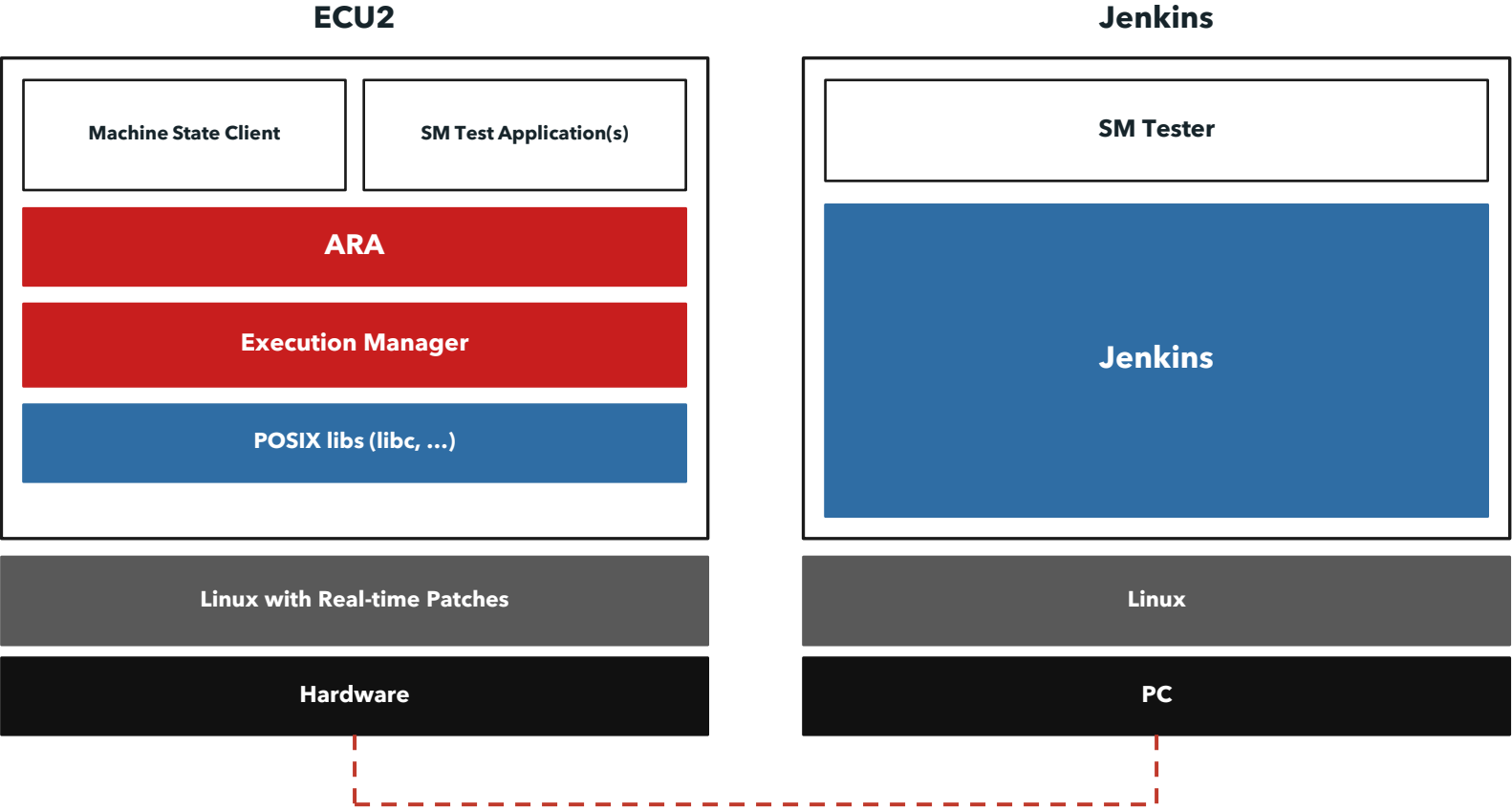
Figure 16.1 · [STS_HM] / STC-PHM-00001



Functional clusters involved: Platform Health Management (local supervision · Watchdog · NV memory)

State Management — test setup

Figure 17.1 · [STS_SM]



Functional clusters involved: State Management + Execution Management (Execution Manager · Machine State Client); coordination via Diagnostics

Key takeaways

The Adaptive Platform in one view

HPC, not deeply-embedded

AP targets compute-heavy, updatable ADAS/AD functions on Ethernet + manycore.

Service-oriented to the core

ara::com Proxy/Skeleton with methods, events, fields, triggers; location-transparent.

Functional clusters

Runtime (EM/SM), Comm, Storage, Security, PHM, UCM, Diagnostics – clear responsibilities.

Safety built in

PHM supervision (alive/deadline/logical) + watchdog feed recovery via SM.

Design vs deployment

Abstract application design bound to hardware by three manifests.

C++ done safely

ara::core Result/ErrorCode (no exceptions), SOLID, acyclic dependencies.

Thank You

Nguyen Nguyen – baonguyenpro@gmail.com

AUTOSAR Adaptive Platform · Compiled from AUTOSAR R22-11 specifications

